TRƯỜNG ĐẠI HỌC CÔNG NGHỆ KỸ THUẬT TP. HỒ CHÍ MINH
KHOA ĐIỆN – ĐIỆN TỬ

MÔN HỌC: PHÁT TRIỂN ỨNG DỤNG DI ĐỘNG

HCMUTE

TIỂU LUẬN CUỐI KÌ

GIẢNG VIÊN HƯỚNG DẪN: Th.S Huỳnh Hoàng Hà

MÃ HỌC PHẦN VÀ MÃ LỚP: MBAD436364_01

Tp. Hồ Chí Minh, tháng 10 năm 2026

TRƯỜNG ĐẠI HỌC CÔNG NGHỆ KỸ THUẬT TP. HỒ CHÍ MINH
KHOA ĐIỆN – ĐIỆN TỬ

MÔN HỌC: PHÁT TRIỂN ỨNG DỤNG DI ĐỘNG

HCMUTE

TIỂU LUẬN CUỐI KÌ

XÂY DỰNG ỨNG DỤNG DI ĐỘNG CHẤM CÔNG BẰNG VÂN TAY KẾT HỢP ESP32 VÀ FIREBASE

GVHD: Th.S Huỳnh Hoàng Hà

Sinh viên thực hiện:

1. Trần Dương Công Vinh - 24119216
2. Yín Hữu Duy - 24119121
3. Trần Hồng Anh Thi - 24119198
4. Lê Tiến Thịnh - 24119200
5. Võ Ngọc Trai - 24119206

Mã lớp học: MBAD436364_01

Tp. Hồ Chí Minh, tháng 10 năm 2026

MỤC LỤC

MỤC LỤC **3**
CHƯƠNG 1: GIỚI THIỆU **1**
1.1 Lý do chọn đề tài và tính cấp thiết 1
1.2 Mục tiêu nghiên cứu 2
1.2.1 Mục tiêu tổng quát 2
1.2.2 Mục tiêu cụ thể 2
1.3 Đối tượng và phạm vi nghiên cứu 3
1.3.1 Đối tượng nghiên cứu 3
1.3.2 Phạm vi nghiên cứu 3
1.4 Phương pháp nghiên cứu 4
1.4.1 Phương pháp nghiên cứu tài liệu 5
1.4.2 Phương pháp phân tích yêu cầu 5
1.4.3 Phương pháp thiết kế hệ thống 5
1.4.4 Phương pháp phát triển ứng dụng 5
1.4.5 Phương pháp tích hợp hệ thống 5
1.4.6 Phương pháp thực nghiệm và kiểm thử 5
1.5 Ý nghĩa khoa học và thực tiễn 6
1.5.1 Ý nghĩa khoa học 6
1.5.2 Ý nghĩa thực tiễn 6
1.6 Bố cục đề tài 7
CHƯƠNG 2: CƠ SỞ LÝ THUYẾT **8**
2.1 Cơ sở lý thuyết về hệ thống chấm công bằng vân tay 8
2.1.1 Khái niệm sinh trắc học và nhận dạng vân tay 8
2.1.2 Nguyên lý nhận dạng vân tay 8
2.1.3 Quy trình chấm công bằng vân tay 9

2.2 Nghiên cứu linh kiện phần cứng chủ chốt 10
2.2.1 Vi điều khiển ESP32 NodeMCU LuaNode32 10
2.2.2 Cảm biến vân tay AS608 11
2.2.3 LED báo trạng thái 12
2.2.4 Buzzer Active 12
2.3 Các giao thức truyền thông liên quan 13
2.3.1 Giao thức truyền thông nối tiếp UART 13
2.3.2 Giao thức Wi-Fi 13
2.3.3 REST API 14
2.4 Firebase và cơ sở dữ liệu thời gian thực 14
2.4.1 Tổng quan về Firebase 14
2.4.2 Firebase Realtime Database 15
2.4.3 Đồng bộ dữ liệu thời gian thực 15
2.4.4 Cấu trúc dữ liệu nhân viên 16
2.5 Phát triển ứng dụng di động 16
2.5.1 React Native 16
2.5.2 TypeScript 17
2.5.3 Giao diện và chức năng ứng dụng 17
2.6 Phần mềm lập trình và môi trường phát triển 18
2.6.1 Môi trường phát triển chương trình ESP32 18
2.6.2 Ngôn ngữ lập trình 18
2.6.3 Môi trường phát triển ứng dụng di động 19
CHƯƠNG 3: PHÂN TÍCH VÀ THIẾT KẾ HỆ THỐNG 20
3.1 Phân tích và thiết kế tổng thể 20
3.1.1 Phân tích yêu cầu chức năng 20
3.1.2 Phân tích yêu cầu phi chức năng 21

3.1.3 Kiến trúc tổng thể hệ thống 22
3.1.4 Luồng dữ liệu tổng quát 23
3.2 Thiết kế phần cứng và thiết bị chấm công 24
3.2.1 Thiết kế phần cứng ESP32 24
3.2.2 Thiết kế kết nối ESP32 và AS608 24
3.2.3 Thiết kế hệ thống chỉ báo 25
3.3 Thiết kế phần mềm ESP32 25
3.3.1 Kiến trúc phần mềm ESP32 25
3.3.2 Thiết kế chức năng đăng ký vân tay 26
3.3.3 Thiết kế chức năng nhận dạng và xử lý chấm công 26
3.3.4 Thiết kế đồng bộ thời gian 27
3.3.5 Thiết kế giao tiếp Firebase REST 27
3.4 Thiết kế cơ sở dữ liệu và ứng dụng di động 28
3.4.1 Thiết kế cơ sở dữ liệu Firebase 28
3.4.2 Thiết kế xác thực và phân quyền 29
3.4.3 Thiết kế cấu trúc ứng dụng React Native 29
3.4.4 Thiết kế giao diện đăng nhập 30
3.4.5 Thiết kế Dashboard Admin 31
3.4.6 Thiết kế Dashboard Employee 32
3.4.7 Thiết kế chức năng quản lý nhân viên 33
3.4.8 Thiết kế chức năng đăng ký vân tay 34
3.4.9 Thiết kế quản lý ca làm việc 34
3.4.10 Thiết kế lịch sử chấm công và báo cáo 35
3.4.11 Thiết kế cơ chế đồng bộ dữ liệu trong ứng dụng 36
3.5 Thiết kế luồng nghiệp vụ chính 37

3.5.1 Luồng đăng nhập 37
3.5.2 Luồng Check-in 37
3.5.3 Luồng Check-out 37
3.5.4 Luồng đăng ký vân tay 38
3.5.5 Luồng xóa nhân viên 38
3.6 Thiết kế điều hướng ứng dụng.....38
CHƯƠNG 4: XÂY DỰNG, TÍCH HỢP VÀ KIỂM THỬ HỆ THỐNG 39
4.1 Xây dựng hệ thống phần cứng.....39
4.1.1 Lắp ráp ESP32 và AS608 39
4.1.2 Kết nối LED và buzzer 39
4.1.3 Lập trình ESP32.....40
4.1.4 Xây dựng chức năng nhận dạng và chấm công 41
4.2 Xây dựng ứng dụng di động và cơ sở dữ liệu 42
4.2.1 Xây dựng giao diện ứng dụng.....42
4.2.2 Xây dựng các chức năng của ứng dụng.....43
4.2.3 Xây dựng cơ sở dữ liệu Firebase 44
4.2.4 Kết nối và đồng bộ ứng dụng với Firebase.....44
4.3 Tích hợp và kiểm thử hệ thống.....45
4.3.1 Tích hợp ESP32, AS608 và Firebase.....45
4.3.2 Tích hợp Firebase và ứng dụng Mobile.....45
4.3.3 Kiểm thử chức năng chấm công 46
4.3.4 Kiểm thử chức năng ứng dụng Mobile.....47
4.4 Kết quả và đánh giá 47
4.4.1 Kết quả hoạt động của thiết bị chấm công.....47
4.4.2 Kết quả hoạt động của ứng dụng Mobile.....48
4.4.3 Kết quả kết nối và đồng bộ dữ liệu.....48

4.4.4 Đánh giá tổng thể hệ thống 49
CHƯƠNG 5: KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN 51
5.1 Kết luận 51
5.1.1 Về phần cứng và hệ thống nhúng 51
5.1.2 Về cơ sở dữ liệu và đồng bộ 51
5.1.3 Về ứng dụng di động 51
5.1.4 Về kiểm thử và độ tin cậy 51
5.2 Các hạn chế còn tồn tại 52
5.2.1 Về bảo mật và phân quyền dữ liệu 52
5.2.2 Về tính ổn định khi mất kết nối mạng 52
5.2.3 Về cơ chế thông báo trên ứng dụng di động 52
5.2.4 Về quy mô kiểm thử 52
5.3 Hướng phát triển trong tương lai 53
5.3.1 Hoàn thiện hệ thống bảo mật 53
5.3.2 Nâng cấp khả năng hoạt động ngoại tuyến (Offline Mode) 53
5.3.3 Tích hợp thông báo đẩy (Push Notification) 53
5.3.4 Mở rộng nền tảng và kiểm thử hiệu năng 53

DANH MỤC BẢNG BIỂU, HÌNH ẢNH

***Danh mục hình ảnh**

Hình 2.1: ESP32 NodeMCU LuaNode32 10
Hình 2.2: Module AS608 12
Hình 3.1: Use Case tổng quát của hệ thống.....21
Hình 3.2: Sơ đồ kiến trúc tổng thể hệ thống.....22
Hình 3.3: Sơ đồ luồng dữ liệu ESP32 → Firebase → Mobile App 23
Hình 3.4: Sơ đồ kết nối phần cứng.....24
Hình 3.5: Sơ đồ tuần tự quá trình đăng ký vân tay.....26
Hình 3.6: Sơ đồ cấu trúc Firebase Realtime Database 29
Hình 3.7: Giao diện Login.....31
Hình 3.8: Giao diện Dashboard Admin.....32
Hình 3.9: Giao diện Dashboard Employee.....33
Hình 3.10: Màn hình quản lí danh sách nhân viên.....34
Hình 3.11: Màn hình quản lí ca làm việc 35
Hình 3.12: Giao diện lịch sử chấm công và giao diện báo cáo 36

***Danh mục bảng biểu**

Bảng 2.1: Các thông số kỹ thuật của ESP32 NodeMCU LuaNode32 11
Bảng 2.2: Bảng ra chân của cảm biến vân tay AS608 12
Bảng 4.1: Trạng thái LED và Buzzer của thiết bị 40
Bảng 4.2: Kết quả kiểm thử chức năng chấm công.....46

CHƯƠNG 1: GIỚI THIỆU

1.1 Lý do chọn đề tài và tính cấp thiết

Trong bối cảnh chuyển đổi số đang được ứng dụng ngày càng rộng rãi trong hoạt động quản lý, việc tự động hóa quá trình ghi nhận và quản lý thời gian làm việc có ý nghĩa quan trọng đối với các cơ quan, doanh nghiệp và tổ chức. Công tác chấm công theo phương pháp thủ công hoặc dựa trên các phương thức xác nhận truyền thống có thể phát sinh nhiều hạn chế như mất thời gian tổng hợp dữ liệu, khó theo dõi lịch sử chấm công và khó cung cấp thông tin kịp thời cho người quản lý cũng như người lao động.

Sự phát triển của công nghệ sinh trắc học, hệ thống nhúng, điện toán đám mây và phát triển ứng dụng di động tạo điều kiện để xây dựng các hệ thống chấm công có khả năng tự động hóa quá trình xác thực và quản lý dữ liệu. Trong đó, phương pháp nhận dạng bằng vân tay cho phép xác định người dùng dựa trên đặc điểm sinh trắc học, đồng thời hạn chế sự phụ thuộc vào các phương thức xác thực thông thường.

Từ yêu cầu trên, đề tài "Xây dựng ứng dụng di động chấm công bằng vân tay kết hợp ESP32 và Firebase" được thực hiện với định hướng xây dựng một hệ thống gồm trạm chấm công sử dụng ESP32 và cảm biến vân tay AS608, nền tảng Firebase làm trung gian lưu trữ và đồng bộ dữ liệu, cùng ứng dụng di động Smart Attendance phục vụ việc quản lý và theo dõi thông tin chấm công.

Trong phạm vi môn học Phát triển ứng dụng di động, trọng tâm của đề tài là quá trình phân tích, thiết kế và xây dựng ứng dụng Smart Attendance bằng React Native và TypeScript. Ứng dụng được thiết kế theo hai nhóm người dùng gồm Admin và Employee, cung cấp các chức năng tương ứng với quyền hạn của từng nhóm. Đối với Admin, ứng dụng hỗ trợ quản lý nhân viên, đăng ký liên kết vân tay, quản lý ca làm việc, theo dõi lịch sử chấm công và xem báo cáo, thống kê. Đối với Employee, ứng dụng cung cấp khả năng theo dõi thông tin cá nhân, ca làm việc, trạng thái chấm công và lịch sử chấm công.

Dữ liệu chấm công được tạo ra tại trạm phần cứng và truyền đến Firebase thông qua ESP32. Ứng dụng Smart Attendance khai thác dữ liệu được lưu trữ trên Firebase để hiển thị và cập nhật trạng thái chấm công theo thời gian thực. Khi ứng dụng đang được mở, dữ liệu chấm công mới có thể được cập nhật và hiển thị trên giao diện. Tuy nhiên,

trong phiên bản hiện tại, hệ thống chưa triển khai cơ chế Push Notification thông qua Firebase Cloud Messaging để gửi thông báo đến thanh thông báo của thiết bị khi ứng dụng đang chạy nền hoặc đã đóng. Đây là một giới hạn được xác định trong phạm vi triển khai hiện tại và có thể được xem xét như một hướng phát triển trong tương lai.

Với định hướng trên, đề tài không chỉ tập trung vào việc xây dựng một thiết bị chấm công mà còn nghiên cứu quá trình tích hợp giữa hệ thống nhúng, cơ sở dữ liệu thời gian thực và ứng dụng di động. Qua đó, đề tài tạo cơ sở để vận dụng các kiến thức về phát triển ứng dụng di động vào một bài toán có tính liên kết giữa phần cứng và phần mềm.

1.2 Mục tiêu nghiên cứu

1.2.1 Mục tiêu tổng quát

Mục tiêu tổng quát của đề tài là phân tích, thiết kế và xây dựng ứng dụng di động Smart Attendance phục vụ quản lý và theo dõi dữ liệu chấm công bằng vân tay. Ứng dụng được phát triển bằng React Native và TypeScript, kết nối với hệ thống Firebase để xác thực người dùng, lưu trữ, truy xuất và đồng bộ dữ liệu chấm công.

Bên cạnh việc xây dựng ứng dụng di động, đề tài thực hiện tích hợp ứng dụng với hệ thống chấm công sử dụng ESP32 và AS608 nhằm hình thành một quy trình hoàn chỉnh từ xác thực vân tay, ghi nhận dữ liệu, lưu trữ trên Firebase đến hiển thị kết quả trên thiết bị di động.

1.2.2 Mục tiêu cụ thể

Trên cơ sở mục tiêu tổng quát, đề tài tập trung phân tích và xây dựng ứng dụng Smart Attendance nhằm đáp ứng các yêu cầu quản lý và theo dõi dữ liệu chấm công trong thực tế. Ứng dụng được phát triển trên nền tảng React Native và TypeScript, trong đó chú trọng đến việc xây dựng giao diện, tổ chức các màn hình chức năng, quản lý trạng thái người dùng và kết nối với các dịch vụ Firebase.

Về chức năng, ứng dụng được xây dựng theo hai vai trò người dùng là Admin và Employee. Đối với Admin, hệ thống hướng đến việc hỗ trợ quản lý thông tin nhân viên, đăng ký liên kết vân tay, quản lý ca làm việc, theo dõi lịch sử chấm công và khai thác các thông tin thống kê. Đối với Employee, ứng dụng tập trung vào việc cung cấp thông tin cá nhân, ca làm việc, trạng thái chấm công và lịch sử chấm công của người dùng.

Bên cạnh việc xây dựng các chức năng trên ứng dụng, đề tài hướng đến việc kết nối ứng dụng với Firebase Realtime Database để dữ liệu được lưu trữ và đồng bộ giữa các thành phần của hệ thống. Dữ liệu chấm công được tạo ra từ trạm ESP32 kết hợp với cảm biến vân tay AS608 và được truyền đến Firebase thông qua giao tiếp HTTP/REST API. Từ đó, ứng dụng Smart Attendance có thể truy xuất và hiển thị kết quả chấm công theo thời gian thực khi ứng dụng đang hoạt động.

Cuối cùng, đề tài tiến hành kiểm thử các chức năng chính của ứng dụng và quá trình trao đổi dữ liệu giữa các thành phần nhằm đánh giá mức độ đáp ứng của hệ thống so với các yêu cầu đã đặt ra.

1.3 Đối tượng và phạm vi nghiên cứu

1.3.1 Đối tượng nghiên cứu

Đối tượng nghiên cứu của đề tài bao gồm các thành phần phần cứng, phần mềm và phương thức trao đổi dữ liệu được sử dụng để xây dựng hệ thống chấm công bằng vân tay.

Về phương diện phần cứng, đề tài nghiên cứu vi điều khiển ESP32 NodeMCU LuaNode32, cảm biến vân tay AS608, cùng các thiết bị chỉ báo trạng thái gồm LED xanh, LED đỏ và buzzer. ESP32 tiếp nhận kết quả nhận dạng từ AS608 thông qua giao tiếp UART, xử lý dữ liệu và truyền thông tin chấm công lên Firebase.

Về phương diện phần mềm, đối tượng nghiên cứu chính là ứng dụng di động Smart Attendance, được xây dựng bằng React Native và TypeScript. Ứng dụng thực hiện xác thực người dùng, phân quyền, quản lý thông tin nhân viên, quản lý ca làm việc, theo dõi dữ liệu chấm công và hiển thị báo cáo, thống kê.

Bên cạnh đó, đề tài nghiên cứu Firebase với các thành phần được sử dụng trong hệ thống, bao gồm Firebase Authentication, Firebase Realtime Database và Firebase Storage. Firebase đóng vai trò cung cấp các dịch vụ trực tuyến để xác thực người dùng, lưu trữ dữ liệu và đồng bộ thông tin giữa các thành phần của hệ thống.

1.3.2 Phạm vi nghiên cứu

Đề tài tập trung vào việc xây dựng một ứng dụng di động Android có khả năng kết nối với hệ thống chấm công bằng vân tay thông qua nền tảng Firebase. Trong đó, ứng dụng Smart Attendance là thành phần trọng tâm của đề tài, được phát triển bằng

React Native và TypeScript, với Firebase được sử dụng để thực hiện xác thực người dùng, lưu trữ và đồng bộ dữ liệu.

Về phía ứng dụng, phạm vi nghiên cứu bao gồm các chức năng phục vụ hai nhóm người dùng là Admin và Employee. Ứng dụng hỗ trợ quản lý tài khoản, phân quyền, quản lý nhân viên, liên kết thông tin vân tay, quản lý ca làm việc, theo dõi dữ liệu chấm công và tra cứu lịch sử chấm công. Đối với Employee, ứng dụng cung cấp các thông tin liên quan đến ca làm việc và kết quả chấm công của cá nhân. Các chức năng được xây dựng nhằm bảo đảm người dùng có thể khai thác dữ liệu chấm công phù hợp với vai trò được cấp.

Về phía hệ thống chấm công, đề tài sử dụng ESP32 NodeMCU LuaNode32 kết hợp với cảm biến vân tay AS608. ESP32 tiếp nhận kết quả nhận dạng từ AS608 thông qua giao tiếp UART, sau đó kết nối mạng và truyền dữ liệu đến Firebase bằng HTTP/REST API. Phần cứng được nghiên cứu và tích hợp nhằm cung cấp nguồn dữ liệu đầu vào cho ứng dụng di động, trong khi trọng tâm triển khai của đề tài vẫn là quá trình xây dựng và tích hợp ứng dụng Smart Attendance.

Về phạm vi nền tảng, ứng dụng được triển khai và kiểm thử trên hệ điều hành Android. Firebase Hosting không được sử dụng trong đề tài do hệ thống được xây dựng dưới dạng ứng dụng di động thay vì ứng dụng web. Ngoài ra, phiên bản hiện tại chưa triển khai Firebase Cloud Messaging để gửi Push Notification đến thanh thông báo của thiết bị khi ứng dụng đang chạy nền hoặc đã đóng. Khi ứng dụng được mở, dữ liệu chấm công mới có thể được đồng bộ và hiển thị trên giao diện thông qua cơ chế cập nhật dữ liệu theo thời gian thực.

Những nội dung liên quan đến mở rộng hệ thống, chẳng hạn triển khai thông báo đẩy khi ứng dụng ở trạng thái nền, được xem là hướng phát triển và không thuộc phạm vi triển khai của phiên bản hiện tại.

1.4 Phương pháp nghiên cứu

Đề tài được thực hiện theo phương pháp kết hợp giữa nghiên cứu lý thuyết, phân tích yêu cầu, thiết kế hệ thống, phát triển phần mềm, tích hợp phần cứng và thực nghiệm. Các phương pháp được sử dụng gồm:

1.4.1 Phương pháp nghiên cứu tài liệu

Các tài liệu kỹ thuật liên quan đến ESP32, AS608, giao tiếp UART, Wi-Fi, HTTP/REST API, Firebase và phát triển ứng dụng di động bằng React Native, TypeScript được nghiên cứu nhằm xác định nguyên lý hoạt động và phương thức tích hợp giữa các thành phần.

1.4.2 Phương pháp phân tích yêu cầu

Các yêu cầu của hệ thống được phân tích dựa trên quy trình chấm công thực tế. Từ đó xác định các nhóm người dùng, chức năng tương ứng và dữ liệu cần quản lý. Hai nhóm người dùng chính được xác định là Admin và Employee.

1.4.3 Phương pháp thiết kế hệ thống

Trên cơ sở yêu cầu chức năng, hệ thống được thiết kế theo mô hình gồm trạm chấm công, nền tảng Firebase và ứng dụng di động. Các luồng dữ liệu giữa ESP32, Firebase và Smart Attendance được xác định nhằm bảo đảm dữ liệu chấm công có thể được truyền, lưu trữ và hiển thị phù hợp.

1.4.4 Phương pháp phát triển ứng dụng

Ứng dụng Smart Attendance được xây dựng bằng React Native và TypeScript. Quá trình phát triển tập trung vào việc thiết kế giao diện, xây dựng các màn hình chức năng, xử lý điều hướng, xác thực người dùng, phân quyền và kết nối với Firebase.

1.4.5 Phương pháp tích hợp hệ thống

Ứng dụng di động được tích hợp với Firebase để truy xuất và đồng bộ dữ liệu. Đồng thời, dữ liệu từ trạm chấm công được ESP32 truyền đến Firebase thông qua HTTP/REST API. Việc tích hợp nhằm hình thành luồng dữ liệu từ quá trình xác thực vân tay đến việc hiển thị kết quả trên ứng dụng.

1.4.6 Phương pháp thực nghiệm và kiểm thử

Hệ thống được kiểm thử thông qua các tình huống thực tế, bao gồm đăng nhập, đăng xuất, quản lý nhân viên, đăng ký vân tay, chấm công thành công, chấm công không hợp lệ, Check-in, Check-out, cập nhật dữ liệu và tra cứu lịch sử chấm công.

Các kết quả kiểm thử được ghi nhận nhằm đánh giá mức độ đáp ứng của ứng

dụng đối với các yêu cầu chức năng đã đề ra.

1.5 Ý nghĩa khoa học và thực tiễn

1.5.1 Ý nghĩa khoa học

Về phương diện khoa học và kỹ thuật, đề tài nghiên cứu khả năng tích hợp giữa nhiều thành phần công nghệ trong một hệ thống thống nhất, bao gồm công nghệ sinh trắc học, hệ thống nhúng, cơ sở dữ liệu thời gian thực và phát triển ứng dụng di động.

Đối với lĩnh vực phát triển ứng dụng di động, đề tài tạo điều kiện vận dụng các kiến thức về React Native, TypeScript, thiết kế giao diện, quản lý điều hướng, xác thực người dùng, phân quyền và kết nối dịch vụ đám mây vào một bài toán thực tế.

Việc xây dựng ứng dụng theo hai vai trò Admin và Employee cũng cho phép nghiên cứu cách tổ chức chức năng và dữ liệu theo quyền truy cập của người dùng. Đồng thời, việc sử dụng Firebase Realtime Database giúp nghiên cứu cơ chế đồng bộ dữ liệu giữa ứng dụng và hệ thống máy chủ.

Bên cạnh đó, quá trình tích hợp ứng dụng với ESP32 và AS608 cho thấy khả năng kết hợp giữa ứng dụng di động và hệ thống nhúng, trong đó ứng dụng đóng vai trò là giao diện quản lý và khai thác dữ liệu được tạo ra từ thiết bị chấm công.

1.5.2 Ý nghĩa thực tiễn

Về phương diện thực tiễn, hệ thống cung cấp một phương thức quản lý dữ liệu chấm công tập trung thông qua ứng dụng di động. Thay vì chỉ hiển thị kết quả trực tiếp tại thiết bị chấm công, dữ liệu có thể được lưu trữ trên Firebase và truy xuất thông qua ứng dụng Smart Attendance.

Đối với Admin, ứng dụng hỗ trợ quản lý thông tin nhân viên, ca làm việc, dữ liệu chấm công và báo cáo thống kê. Điều này giúp giảm thao tác tổng hợp dữ liệu thủ công và hỗ trợ theo dõi tình trạng chấm công của nhân viên.

Đối với Employee, ứng dụng cung cấp khả năng theo dõi thông tin ca làm việc, trạng thái chấm công, thời gian Check-in, Check-out, số phút đi trễ, thời gian làm việc và thời gian tăng ca.

Ngoài ra, việc sử dụng cơ chế đồng bộ dữ liệu theo thời gian thực giúp thông tin chấm công mới có thể được cập nhật trên ứng dụng khi ứng dụng đang hoạt động. Tuy nhiên, do phiên bản hiện tại chưa triển khai FCM, hệ thống chưa cung cấp thông báo

đẩy trên thanh thông báo của Android khi ứng dụng đã đóng hoặc đang chạy nền.

1.6 Bố cục đề tài

Nội dung của báo cáo được tổ chức thành năm chương, tập trung vào quá trình phân tích, thiết kế, xây dựng và đánh giá hệ thống, trong đó ứng dụng di động Smart Attendance là thành phần trọng tâm.

Chương 1 – Giới thiệu: Trình bày lý do lựa chọn đề tài, tính cấp thiết của bài toán, mục tiêu nghiên cứu, đối tượng và phạm vi nghiên cứu, phương pháp nghiên cứu, ý nghĩa khoa học và thực tiễn của đề tài.

Chương 2 – Cơ sở lý thuyết: Trình bày các cơ sở lý thuyết liên quan đến hệ thống chấm công bằng vân tay và các công nghệ được sử dụng trong đề tài, bao gồm ESP32, AS608, UART, Wi-Fi, HTTP/REST API, Firebase, React Native và TypeScript.

Chương 3 – Phân tích và thiết kế hệ thống: Phân tích yêu cầu của hệ thống, xác định các tác nhân và chức năng, xây dựng kiến trúc tổng thể, thiết kế cơ sở dữ liệu, thiết kế luồng dữ liệu và thiết kế giao diện của ứng dụng Smart Attendance cho hai nhóm người dùng Admin và Employee.

Chương 4 – Xây dựng, tích hợp và kiểm thử hệ thống: Trình bày quá trình hiện thực ứng dụng, tích hợp Firebase, kết nối với hệ thống ESP32 – AS608, xây dựng các chức năng chính, thực hiện kiểm thử và đánh giá kết quả hoạt động của hệ thống.

Chương 5 – Kết luận và hướng phát triển: Tổng kết những kết quả đạt được so với mục tiêu ban đầu, phân tích các hạn chế còn tồn tại và đề xuất những hướng phát triển tiếp theo của hệ thống.

Bên cạnh năm chương chính, báo cáo có thể kèm theo Tài liệu tham khảo và Phụ lục nhằm cung cấp các tài liệu kỹ thuật, mã nguồn, cấu trúc cơ sở dữ liệu, sơ đồ phần cứng và các hình ảnh minh họa cần thiết.

CHƯƠNG 2: CƠ SỞ LÝ THUYẾT

2.1 Cơ sở lý thuyết về hệ thống chấm công bằng vân tay

2.1.1 Khái niệm sinh trắc học và nhận dạng vân tay

Sinh trắc học là lĩnh vực nghiên cứu và ứng dụng các đặc điểm sinh học hoặc đặc điểm hành vi của con người để nhận dạng và xác thực danh tính. Các đặc điểm sinh trắc học thường có tính tương đối ổn định và có khả năng phân biệt giữa các cá thể khác nhau. Trong các phương pháp sinh trắc học, nhận dạng vân tay là một phương pháp được sử dụng phổ biến trong các hệ thống xác thực tự động.

Vân tay của mỗi người có các đặc điểm hình thái tương đối riêng biệt, được hình thành từ các đường vân trên bề mặt đầu ngón tay. Các đặc điểm này có thể được cảm biến thu nhận và chuyển thành dữ liệu phục vụ cho quá trình so sánh với mẫu vân tay đã được đăng ký trước đó.

Trong hệ thống Smart Attendance, công nghệ nhận dạng vân tay được sử dụng để xác định nhân viên trước khi thực hiện thao tác chấm công. Người dùng không cần nhập mã nhân viên trực tiếp trên ứng dụng mà thực hiện xác thực tại trạm phần cứng thông qua cảm biến AS608. Sau khi nhận dạng thành công, cảm biến trả về mã định danh vân tay tương ứng để ESP32 tiếp tục xử lý và liên kết với thông tin nhân viên được lưu trữ trên Firebase.

Cách tiếp cận này giúp tách quá trình xác thực danh tính khỏi ứng dụng di động. Ứng dụng Smart Attendance tập trung vào việc quản lý thông tin nhân viên, theo dõi lịch làm việc, tra cứu dữ liệu chấm công và hiển thị trạng thái chấm công, trong khi việc thu nhận và nhận dạng vân tay được thực hiện tại thiết bị phần cứng.

2.1.2 Nguyên lý nhận dạng vân tay

Quá trình nhận dạng vân tay trong hệ thống được thực hiện dựa trên sự phối hợp giữa cảm biến AS608 và vi điều khiển ESP32. AS608 đảm nhiệm việc thu nhận và xử lý mẫu vân tay, trong khi ESP32 đảm nhiệm việc điều khiển cảm biến, tiếp nhận kết quả và thực hiện các bước xử lý tiếp theo.

Khi người dùng đặt ngón tay lên cảm biến, AS608 tiến hành thu nhận dữ liệu vân tay. Dữ liệu này được module xử lý và so sánh với các mẫu vân tay đã được đăng ký trong bộ nhớ của cảm biến. Nếu tìm thấy mẫu phù hợp, AS608 trả về mã định danh

tương ứng. ESP32 nhận mã định danh này thông qua giao tiếp UART.

Mã định danh vân tay sau đó được sử dụng để đối chiếu với thông tin nhân viên trên Firebase. Khi xác định được nhân viên tương ứng, hệ thống tiếp tục xử lý trạng thái chấm công và tạo hoặc cập nhật dữ liệu chấm công của nhân viên.

Như vậy, AS608 không chỉ đóng vai trò là thiết bị thu nhận vân tay mà còn thực hiện phần xử lý nhận dạng tại trạm chấm công. ESP32 nhận kết quả nhận dạng dưới dạng mã định danh và sử dụng mã này trong quá trình xử lý nghiệp vụ. Cách tổ chức này giúp giảm lượng dữ liệu cần truyền giữa thiết bị phần cứng và máy chủ.

2.1.3 Quy trình chấm công bằng vân tay

Quy trình chấm công bắt đầu khi nhân viên thực hiện xác thực tại trạm chấm công bằng cách đặt ngón tay lên cảm biến AS608. Sau khi nhận dạng thành công, AS608 trả về mã vân tay tương ứng cho ESP32 thông qua giao tiếp UART.

ESP32 sử dụng mã vân tay để xác định nhân viên tương ứng trên hệ thống dữ liệu. Sau khi xác định được nhân viên, hệ thống kiểm tra trạng thái chấm công hiện tại để xác định thao tác cần thực hiện. Trong hệ thống, hai trạng thái chính được sử dụng là Check-in và Check-out.

Lượt quét đầu tiên trong ngày được xử lý theo trạng thái Check-in. Sau khi nhân viên đã thực hiện Check-in, thao tác Check-out được cho phép sau khoảng thời gian tối thiểu được quy định trong hệ thống. Theo thiết kế hiện tại, thời điểm Check-out phải cách thời điểm Check-in ít nhất 5 phút.

Sau khi xác định trạng thái chấm công, ESP32 truyền dữ liệu lên Firebase Realtime Database thông qua kết nối Wi-Fi và REST API. Dữ liệu sau đó được ứng dụng Smart Attendance đồng bộ và hiển thị thông qua cơ chế lắng nghe dữ liệu thời gian thực.

Tại trạm phần cứng, hệ thống đồng thời sử dụng LED và buzzer để cung cấp phản hồi trực tiếp cho người dùng. LED xanh được sử dụng để biểu thị kết quả thành công, trong khi LED đỏ biểu thị trường hợp thất bại. Buzzer được sử dụng để tạo tín hiệu âm thanh phản hồi trong cả hai trường hợp.

2.2 Nghiên cứu linh kiện phần cứng chủ chốt

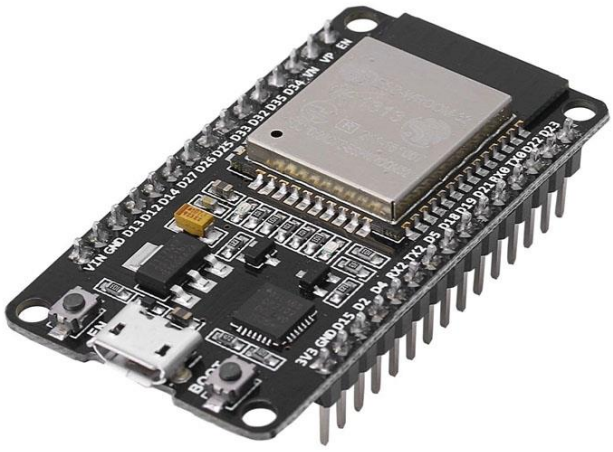
2.2.1 Vi điều khiển ESP32 NodeMCU LuaNode32

ESP32 NodeMCU LuaNode32 được sử dụng làm bộ xử lý trung tâm của trạm chấm công. Trong hệ thống, ESP32 chịu trách nhiệm điều khiển cảm biến AS608, xử lý kết quả nhận dạng, điều khiển các thiết bị chỉ thị, kết nối mạng Wi-Fi và trao đổi dữ liệu với Firebase.

ESP32 đóng vai trò là thành phần trung gian giữa lớp thiết bị và lớp dịch vụ dữ liệu. Dữ liệu nhận dạng từ AS608 được ESP32 tiếp nhận thông qua giao tiếp UART. Sau khi xử lý, ESP32 sử dụng kết nối Wi-Fi để trao đổi dữ liệu với Firebase Realtime Database thông qua REST API.

Bên cạnh đó, ESP32 còn đảm nhiệm việc điều khiển LED xanh, LED đỏ và buzzer nhằm phản hồi trạng thái xử lý trực tiếp tại trạm chấm công. Chương trình điều khiển ESP32 được tổ chức thành nhiều module riêng biệt như xử lý chấm công, quản lý đăng ký vân tay, giao tiếp cảm biến, Wi-Fi, Firebase, đồng hồ thời gian và điều khiển chỉ thị.

Cấu trúc module hóa giúp tách biệt các chức năng trong chương trình, qua đó tạo điều kiện thuận lợi cho việc phát triển, kiểm thử và bảo trì hệ thống.



Hình 2.1: ESP32 NodeMCU LuaNode32

Bảng 2.1: Các thông số kỹ thuật của ESP32 NodeMCU LuaNode32

Thành phần	Thông số chi tiết
Vi điều khiển	NodeMCU ESP32
Vi xử lý	Tensilica LX6 Dual-Core, 240 MHz
Bộ nhớ	512 kB SRAM / 4 MB Flash
Kết nối không dây (WiFi)	Wi-Fi 802.11 b/g/n (2.4 GHz)
Giao tiếp ngoại vi	UART, I2C, SPI, ADC
Điện áp hoạt động	3.3V (Cấp nguồn qua cổng 5V MicroUSB)

2.2.2 Cảm biến vân tay AS608

AS608 là module cảm biến vân tay được sử dụng để thực hiện chức năng thu nhận và nhận dạng vân tay trong hệ thống. Module được sử dụng trong đề tài là AS608 – 500 DPI.

Trong hệ thống Smart Attendance, AS608 đảm nhiệm phần thu nhận và xử lý nhận dạng vân tay. Khi người dùng đặt ngón tay lên cảm biến, module thực hiện quá trình thu nhận và so sánh với các mẫu vân tay đã được đăng ký. Nếu quá trình xác thực thành công, module cung cấp mã định danh của mẫu vân tay cho ESP32.

AS608 được kết nối với ESP32 thông qua giao tiếp UART. Mô hình trao đổi dữ liệu được thực hiện theo nguyên tắc ESP32 gửi lệnh điều khiển đến AS608 và nhận kết quả xử lý từ module. Mã định danh nhận được sau quá trình nhận dạng được ESP32 sử dụng để xác định nhân viên tương ứng trong hệ thống dữ liệu.

Ngoài chức năng nhận dạng khi chấm công, AS608 còn được sử dụng trong quá trình đăng ký vân tay cho nhân viên. Khi quản trị viên thực hiện đăng ký vân tay thông qua ứng dụng, yêu cầu đăng ký được truyền đến trạm phần cứng để thực hiện quá trình thu nhận và lưu mẫu vân tay.



Hình 2.2: Module AS608

Bảng 2.2: Bảng ra chân của cảm biến vân tay AS608

STT chân	Tên chân (Pin)
Pin 1	3.3V DC
Pin 2	TXD
Pin 3	RXD
Pin 4	GND
Pin 5	TOUCH
Pin 6	VA
Pin 7	D+
Pin 8	D-

2.2.3 LED báo trạng thái

LED được sử dụng làm thành phần hiển thị trạng thái trực tiếp tại trạm chấm công. Hệ thống sử dụng hai LED với hai màu khác nhau để phân biệt kết quả xử lý.

LED xanh được sử dụng để biểu thị quá trình xác thực và chấm công thành công. Ngược lại, LED đỏ được sử dụng để biểu thị trường hợp xác thực hoặc chấm công thất bại.

Việc sử dụng LED giúp người dùng nhận biết kết quả ngay tại trạm chấm công mà không cần quan sát ứng dụng di động. Thành phần này đặc biệt hữu ích trong trường hợp người dùng chỉ tương tác trực tiếp với thiết bị phần cứng.

Trong mô hình hiện tại, LED xanh được điều khiển bởi GPIO18 của ESP32 và LED đỏ được điều khiển bởi GPIO19.

2.2.4 Buzzer Active

Buzzer được sử dụng để tạo tín hiệu âm thanh phản hồi cho người dùng tại trạm

chấm công. Theo linh kiện thực tế của đề tài, buzzer được sử dụng là loại Active.

Buzzer được điều khiển bởi ESP32 và được kích hoạt khi hệ thống phát sinh kết quả xử lý chấm công. Trong hệ thống hiện tại, buzzer được sử dụng cho cả trường hợp chấm công thành công và chấm công thất bại. Việc kết hợp tín hiệu âm thanh với LED giúp người dùng nhận biết trạng thái của quá trình xử lý một cách trực quan hơn.

Buzzer được điều khiển thông qua GPIO4 của ESP32.

2.3 Các giao thức truyền thông liên quan

2.3.1 Giao thức truyền thông nối tiếp UART

UART (Universal Asynchronous Receiver-Transmitter) là phương thức truyền thông nối tiếp không đồng bộ, được sử dụng phổ biến trong các hệ thống nhúng để trao đổi dữ liệu giữa các thiết bị.

Trong hệ thống Smart Attendance, UART được sử dụng làm giao tiếp giữa ESP32 và cảm biến vân tay AS608. ESP32 gửi các lệnh điều khiển đến AS608 và nhận kết quả xử lý từ module thông qua hai đường truyền TX và RX.

Kết nối tín hiệu giữa hai thiết bị được thực hiện theo nguyên tắc:

TX của AS608 → RX của ESP32.

RX của AS608 → TX của ESP32.

Theo cấu hình thực tế của hệ thống, AS608 sử dụng tốc độ truyền UART 57.600 bps. Chân TX của AS608 được kết nối với GPIO16 của ESP32 và chân RX của AS608 được kết nối với GPIO17 của ESP32.

Việc sử dụng UART giúp tách biệt quá trình giao tiếp giữa cảm biến và vi điều khiển khỏi quá trình truyền dữ liệu lên máy chủ. AS608 thực hiện xử lý nhận dạng và trả về kết quả, còn ESP32 tiếp nhận kết quả này để thực hiện các chức năng nghiệp vụ tiếp theo.

2.3.2 Giao thức Wi-Fi

Wi-Fi được sử dụng làm phương thức kết nối mạng cho ESP32. Sau khi kết nối thành công với mạng không dây, ESP32 có khả năng trao đổi dữ liệu với các dịch vụ trực tuyến thông qua Internet.

Trong hệ thống Smart Attendance, Wi-Fi đóng vai trò là cầu nối giữa trạm chấm công và Firebase. Sau khi nhận được kết quả nhận dạng từ AS608, ESP32 sử dụng kết

nối Wi-Fi để đọc hoặc cập nhật dữ liệu trên Firebase Realtime Database.

Việc sử dụng Wi-Fi giúp trạm chấm công không cần kết nối trực tiếp với điện thoại của người dùng. Dữ liệu được truyền thông qua Firebase, từ đó cho phép ứng dụng Smart Attendance truy xuất và đồng bộ dữ liệu từ máy chủ.

2.3.3 REST API

REST (Representational State Transfer) là một kiến trúc được sử dụng để xây dựng các dịch vụ giao tiếp thông qua giao thức HTTP/HTTPS. Trong hệ thống của đề tài, REST API được sử dụng làm phương thức giao tiếp giữa ESP32 và Firebase Realtime Database.

Module FirebaseREST trong chương trình ESP32 thực hiện việc xây dựng địa chỉ truy cập đến dữ liệu Firebase và gửi các HTTP request đến máy chủ. Theo mã nguồn thực tế, module hỗ trợ các phương thức HTTP phục vụ việc đọc và cập nhật dữ liệu, trong đó GET được sử dụng để đọc dữ liệu và PATCH được sử dụng để cập nhật một phần dữ liệu tại đường dẫn tương ứng.

Dữ liệu trao đổi được thực hiện dưới dạng JSON. Khi gửi request, chương trình thiết lập kiểu nội dung application/json và thực hiện giao tiếp thông qua HTTP client.

Cơ chế REST API tạo ra một lớp giao tiếp giữa chương trình ESP32 và Firebase mà không yêu cầu ESP32 phải sử dụng trực tiếp thư viện giao diện của ứng dụng di động. Nhờ đó, thiết bị phần cứng và ứng dụng có thể cùng truy cập vào một nguồn dữ liệu trung tâm nhưng vẫn duy trì sự tách biệt giữa hai thành phần.

2.4 Firebase và cơ sở dữ liệu thời gian thực

2.4.1 Tổng quan về Firebase

Firebase là nền tảng dịch vụ được sử dụng để cung cấp các thành phần backend cho ứng dụng. Trong hệ thống Smart Attendance, Firebase đóng vai trò là lớp dịch vụ trung gian giữa trạm chấm công và ứng dụng di động.

Các thành phần Firebase được sử dụng trong đề tài bao gồm Firebase Authentication, Firebase Realtime Database và Firebase Storage. Firebase Authentication phục vụ việc xác thực tài khoản người dùng; Realtime Database lưu trữ và đồng bộ dữ liệu chấm công, nhân viên và ca làm việc; Firebase Storage được sử dụng để lưu trữ dữ liệu hình ảnh đại diện.

Việc sử dụng Firebase giúp ứng dụng di động không cần xây dựng một máy chủ backend riêng cho các chức năng cơ bản của hệ thống. Đồng thời, dữ liệu từ ESP32 có thể được cập nhật lên Firebase và được ứng dụng truy xuất thông qua các cơ chế tương ứng.

2.4.2 Firebase Realtime Database

Firebase Realtime Database là cơ sở dữ liệu dạng NoSQL, trong đó dữ liệu được tổ chức theo cấu trúc cây. Trong hệ thống Smart Attendance, Realtime Database là thành phần trung tâm để lưu trữ thông tin liên quan đến người dùng, nhân viên, ca làm việc và dữ liệu chấm công.

Một số nhóm dữ liệu chính được sử dụng trong hệ thống gồm thông tin tài khoản tại /users, thông tin nhân viên tại /employees, thông tin ca làm việc tại /shifts và dữ liệu chấm công tại /attendance.

Bên cạnh dữ liệu phục vụ ứng dụng, hệ thống còn sử dụng các nhóm dữ liệu phục vụ quá trình tích hợp giữa ứng dụng và thiết bị phần cứng như /attendance_logs, /enrollment_requests, /devices và /settings.

Cấu trúc dữ liệu được tổ chức theo các đường dẫn riêng giúp các thành phần trong hệ thống có thể truy cập đến đúng nhóm dữ liệu cần thiết. Chi tiết cấu trúc cơ sở dữ liệu và mối quan hệ giữa các node sẽ được trình bày trong Chương 3.

2.4.3 Đồng bộ dữ liệu thời gian thực

Một trong những đặc điểm quan trọng của Firebase Realtime Database được sử dụng trong đề tài là khả năng theo dõi sự thay đổi của dữ liệu. Ứng dụng Smart Attendance sử dụng cơ chế listener để theo dõi các node dữ liệu cần thiết.

Trong ứng dụng React Native, các màn hình Dashboard và dữ liệu chấm công có thể đăng ký listener thông qua onValue(ref(...), callback). Khi dữ liệu tương ứng trên Firebase thay đổi, callback được kích hoạt và ứng dụng cập nhật lại trạng thái giao diện.

Theo đó, khi ESP32 ghi nhận một sự kiện chấm công và cập nhật dữ liệu lên Firebase, ứng dụng có thể nhận được dữ liệu mới và cập nhật giao diện mà không yêu cầu người dùng tải lại toàn bộ màn hình.

Cần phân biệt cơ chế đồng bộ dữ liệu thời gian thực với Push Notification. Trong phiên bản hiện tại của hệ thống, ứng dụng cập nhật và hiển thị kết quả chấm công khi

ứng dụng đang hoạt động; hệ thống chưa triển khai cơ chế thông báo Push đến thanh thông báo Android khi ứng dụng chạy nền hoặc đã đóng.

Cơ chế listener cũng cần được quản lý vòng đời phù hợp. Khi màn hình hoặc thành phần giao diện không còn sử dụng listener, listener cần được hủy đăng ký để tránh việc nhận dữ liệu không cần thiết hoặc cập nhật vào thành phần giao diện đã bị đóng.

2.4.4 Cấu trúc dữ liệu nhân viên

Thông tin nhân viên được lưu trữ trên Firebase nhằm phục vụ cả chức năng quản lý nhân sự và liên kết với dữ liệu vân tay tại trạm chấm công.

Trong hệ thống hiện tại, node /employees/{employeeId} chứa các thông tin chính như mã nhân viên, họ tên, phòng ban, mã vân tay, ca làm việc được phân công và trạng thái hoạt động. Trong đó, fingerprintId giữ vai trò quan trọng trong việc liên kết giữa nhân viên trên Firebase với mã vân tay được AS608 nhận dạng.

Bên cạnh thông tin nhân viên, node /users/{uid} được sử dụng để quản lý tài khoản đăng nhập và vai trò của người dùng. Thông tin vai trò được sử dụng để xác định người dùng là quản trị viên hay nhân viên, từ đó ứng dụng điều hướng đến các chức năng tương ứng.

Dữ liệu chấm công được tổ chức tại node /attendance/{date}/{employeeId}, trong đó các thông tin như thời gian Check-in, thời gian Check-out, trạng thái, số phút đi trễ, số giờ làm việc và số giờ tăng ca được lưu trữ để phục vụ việc tra cứu và thống kê.

2.5 Phát triển ứng dụng di động

2.5.1 React Native

React Native là framework được sử dụng để xây dựng ứng dụng di động Smart Attendance. Framework này cho phép phát triển giao diện ứng dụng bằng JavaScript hoặc TypeScript và tổ chức chương trình theo các thành phần giao diện có khả năng tái sử dụng.

Trong đề tài, React Native được sử dụng làm nền tảng chính để xây dựng ứng dụng Smart Attendance trên Android. Ứng dụng đóng vai trò là giao diện quản lý và theo dõi dữ liệu của toàn bộ hệ thống chấm công.

Khác với trạm chấm công, ứng dụng di động không trực tiếp thực hiện quá trình

nhận dạng vân tay. Thay vào đó, ứng dụng tương tác với dữ liệu được lưu trữ trên Firebase. Khi dữ liệu được cập nhật từ ESP32, ứng dụng nhận thay đổi thông qua Firebase Realtime Database và cập nhật giao diện tương ứng.

2.5.2 TypeScript

TypeScript là ngôn ngữ lập trình được sử dụng kết hợp với React Native trong quá trình phát triển ứng dụng Smart Attendance. TypeScript mở rộng JavaScript bằng hệ thống kiểu dữ liệu, giúp mô tả rõ hơn cấu trúc của dữ liệu được sử dụng trong chương trình.

Trong hệ thống, TypeScript được sử dụng để xây dựng các thành phần giao diện, xử lý dữ liệu từ Firebase, định nghĩa kiểu dữ liệu và triển khai các chức năng nghiệp vụ của ứng dụng.

Việc sử dụng TypeScript đặc biệt có ý nghĩa đối với hệ thống có nhiều nhóm dữ liệu như thông tin nhân viên, tài khoản, ca làm việc và bản ghi chấm công. Việc xác định kiểu dữ liệu giúp quá trình phát triển và bảo trì mã nguồn rõ ràng hơn.

2.5.3 Giao diện và chức năng ứng dụng

Ứng dụng Smart Attendance được xây dựng nhằm cung cấp giao diện quản lý và theo dõi hệ thống chấm công. Ứng dụng sử dụng cơ chế xác thực tài khoản bằng Email và Password thông qua Firebase Authentication.

Sau khi đăng nhập, hệ thống xác định vai trò của tài khoản để cung cấp giao diện và chức năng phù hợp. Hai vai trò chính trong ứng dụng là Admin và Employee.

Đối với Admin, ứng dụng cung cấp các chức năng quản lý hệ thống như Dashboard, quản lý nhân viên, thêm nhân viên, chỉnh sửa thông tin nhân viên, xóa nhân viên, đăng ký vân tay cho nhân viên, quản lý ca làm việc, tra cứu lịch sử chấm công, tìm kiếm, lọc dữ liệu và xem các báo cáo thống kê.

Đối với Employee, ứng dụng cung cấp giao diện cá nhân để theo dõi thông tin làm việc. Người dùng có thể xem Dashboard cá nhân, ca làm việc, trạng thái trong ngày và lịch sử chấm công. Các thông tin liên quan đến thời gian Check-in, Check-out, số phút đi trễ, số giờ làm việc và số giờ tăng ca được hiển thị trên ứng dụng.

Khi phát sinh một sự kiện chấm công, ứng dụng đang hoạt động có thể cập nhật dữ liệu từ Firebase và hiển thị kết quả tương ứng trên giao diện. Cơ chế này giúp người

dùng theo dõi trạng thái chấm công mà không cần thực hiện thao tác tải lại dữ liệu thủ công.

Ứng dụng cũng cung cấp chức năng đăng xuất để kết thúc phiên làm việc của tài khoản hiện tại. Việc phân chia chức năng theo vai trò giúp giao diện phù hợp với từng nhóm người dùng và hạn chế việc hiển thị các chức năng không cần thiết.

2.6 Phần mềm lập trình và môi trường phát triển

2.6.1 Môi trường phát triển chương trình ESP32

Chương trình điều khiển ESP32 được tổ chức theo cấu trúc gồm nhiều module chức năng. Từ mã nguồn thực tế của hệ thống có thể xác định các thành phần như AttendanceProcessor,AttendanceRepository,AttendanceService,EnrollmentRepository, EnrollmentService, FingerprintSensor, FirebaseREST, HrRepository, Indicator, NtpClock, WiFiService và main.

Các module này đảm nhiệm những nhóm chức năng khác nhau, bao gồm xử lý chấm công, quản lý đăng ký vân tay, giao tiếp với cảm biến, giao tiếp Firebase, quản lý kết nối Wi-Fi, đồng bộ thời gian và điều khiển các thiết bị chỉ thị.

Cách tổ chức chương trình thành các module riêng biệt giúp giảm sự phụ thuộc giữa các chức năng và thuận tiện hơn cho quá trình phát triển, kiểm thử và bảo trì.

2.6.2 Ngôn ngữ lập trình

Phần mềm điều khiển ESP32 được xây dựng bằng ngôn ngữ C/C++ trên nền tảng Arduino dành cho ESP32.

Ngôn ngữ này được sử dụng để triển khai các chức năng điều khiển cảm biến AS608, giao tiếp UART, điều khiển LED và buzzer, kết nối Wi-Fi, giao tiếp Firebase REST API, xử lý dữ liệu chấm công và quản lý thời gian.

Trong chương trình, các chức năng được phân tách thành những module riêng nhằm tổ chức mã nguồn rõ ràng hơn. Module FingerprintSensor chịu trách nhiệm giao tiếp với cảm biến AS608, WiFiService quản lý kết nối mạng, FirebaseREST xử lý giao tiếp với Firebase, trong khi các module liên quan đến Attendance và Enrollment thực hiện xử lý nghiệp vụ chấm công và đăng ký vân tay.

2.6.3 Môi trường phát triển ứng dụng di động

Ứng dụng Smart Attendance được phát triển bằng React Native 0.87.1 và TypeScript. Môi trường lập trình chính được sử dụng là Visual Studio Code.

Visual Studio Code được sử dụng để xây dựng mã nguồn, tổ chức cấu trúc dự án, chỉnh sửa các thành phần giao diện và thực hiện quá trình phát triển ứng dụng React Native.

Cấu trúc mã nguồn ứng dụng được tổ chức thành các nhóm thành phần như components, screens, services, types và utils. Thành phần services đảm nhiệm việc giao tiếp với Firebase, trong khi các screens tương ứng với các màn hình chức năng của ứng dụng.

Trong quá trình phát triển, ứng dụng được kiểm thử trên môi trường Android phù hợp với phạm vi triển khai của đề tài.

CHƯƠNG 3: PHÂN TÍCH VÀ THIẾT KẾ HỆ THỐNG

3.1 Phân tích và thiết kế tổng thể

3.1.1 Phân tích yêu cầu chức năng

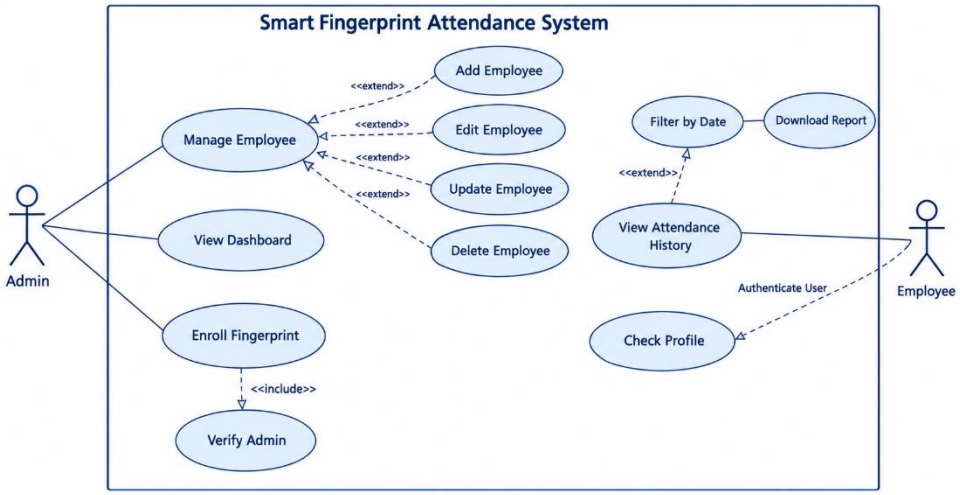
Hệ thống Smart Attendance được xây dựng nhằm tự động hóa quá trình ghi nhận chấm công bằng vân tay, lưu trữ tập trung dữ liệu và cung cấp giao diện di động để quản lý, tra cứu và theo dõi tình trạng làm việc. Dựa trên kiến trúc hệ thống và các nhóm người dùng, các yêu cầu chức năng được phân thành ba nhóm chính gồm chức năng của trạm chấm công, chức năng của quản trị viên và chức năng của nhân viên.

Đối với trạm chấm công, hệ thống phải có khả năng tiếp nhận thao tác đặt ngón tay của người dùng, thực hiện nhận dạng thông qua cảm biến AS608 và trả về mã định danh vân tay cho ESP32. ESP32 có nhiệm vụ xử lý mã định danh, xác định nhân viên tương ứng và thực hiện nghiệp vụ chấm công. Kết quả sau đó được truyền lên Firebase thông qua kết nối Wi-Fi và REST API. Trạm chấm công đồng thời phải cung cấp phản hồi trực tiếp thông qua LED xanh, LED đỏ và buzzer.

Đối với quản trị viên, ứng dụng Smart Attendance cung cấp các chức năng phục vụ quản lý nhân sự và dữ liệu chấm công. Quản trị viên có thể đăng nhập hệ thống, theo dõi Dashboard, quản lý thông tin nhân viên, thêm, chỉnh sửa và xóa nhân viên, thực hiện đăng ký vân tay cho nhân viên, quản lý ca làm việc, tra cứu lịch sử chấm công, tìm kiếm và lọc dữ liệu. Ngoài ra, hệ thống cung cấp chức năng báo cáo và thống kê nhằm hỗ trợ theo dõi tình hình chấm công.

Đối với nhân viên, ứng dụng cung cấp giao diện cá nhân để theo dõi thông tin làm việc. Nhân viên có thể xem Dashboard cá nhân, ca làm việc, trạng thái chấm công trong ngày và lịch sử chấm công. Các thông tin như thời gian Check-in, thời gian Check-out, số phút đi trễ, số giờ làm việc và số giờ tăng ca được hiển thị dựa trên dữ liệu được lưu trữ trên Firebase.

Ứng dụng cũng phải hỗ trợ xác thực tài khoản bằng Email và Password thông qua Firebase Authentication. Sau khi đăng nhập, hệ thống xác định vai trò của tài khoản để điều hướng người dùng đến giao diện tương ứng. Việc đăng xuất phải kết thúc phiên đăng nhập hiện tại và đưa người dùng trở về màn hình xác thực.



Hình 3.1: Use Case tổng quát của hệ thống

3.1.2 Phân tích yêu cầu phi chức năng

Bên cạnh các chức năng nghiệp vụ, hệ thống cần đáp ứng một số yêu cầu phi chức năng liên quan đến khả năng sử dụng, tính ổn định, khả năng bảo trì và tính nhất quán của dữ liệu.

Đối với ứng dụng di động, giao diện cần được tổ chức rõ ràng để người dùng có thể thực hiện các thao tác quản lý và tra cứu một cách thuận tiện. Các trạng thái như đang tải dữ liệu, dữ liệu rỗng, thao tác thành công và lỗi cần được xử lý rõ ràng thay vì để giao diện ở trạng thái không xác định.

Mã nguồn ứng dụng được tổ chức theo hướng module hóa, trong đó các thành phần giao diện, màn hình, dịch vụ Firebase, kiểu dữ liệu và các hàm tiện ích được phân tách tương đối độc lập. Cách tổ chức này giúp thuận tiện cho quá trình phát triển và bảo trì khi số lượng chức năng của ứng dụng tăng lên.

Đối với dữ liệu, hệ thống sử dụng Firebase Realtime Database nhằm đồng bộ thông tin giữa trạm chấm công và ứng dụng. Khi một bản ghi chấm công được cập nhật, ứng dụng có khả năng nhận thay đổi thông qua Realtime Listener và cập nhật lại giao diện tương ứng.

Về bảo mật, việc kiểm tra vai trò ở tầng giao diện chỉ có tác dụng kiểm soát chức năng được hiển thị cho người dùng. Cơ chế bảo vệ dữ liệu thực tế cần được thực hiện tại Firebase Security Rules để giới hạn quyền truy cập theo vai trò và phạm vi dữ liệu. Đây là một nội dung cần được xem xét và hoàn thiện trong quá trình triển khai thực tế.

3.1.3 Kiến trúc tổng thể hệ thống

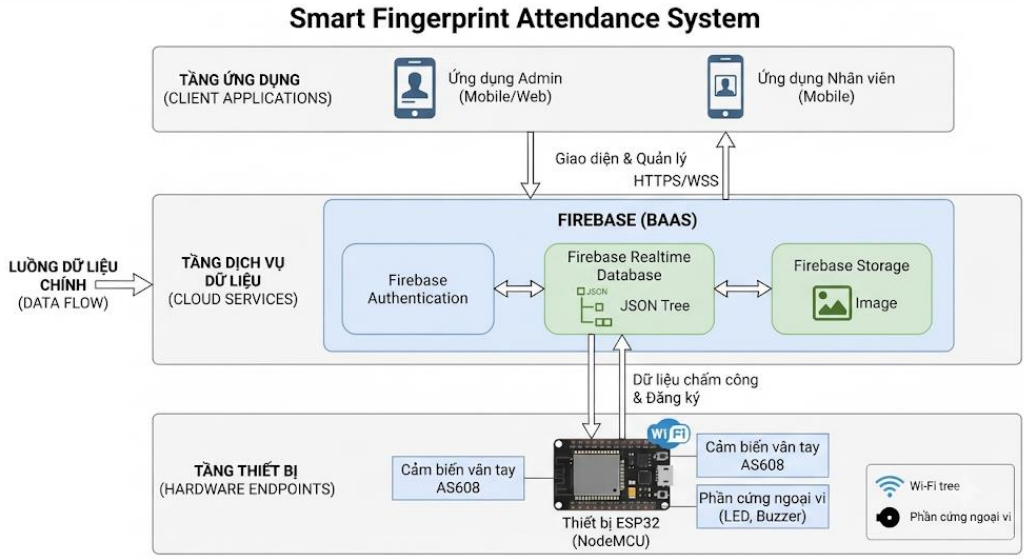
Hệ thống Smart Attendance được thiết kế theo mô hình kiến trúc phân tầng ba lớp, kết hợp với cơ chế xử lý theo sự kiện. Kiến trúc này gồm lớp thiết bị, lớp dịch vụ và lớp ứng dụng.

Lớp thiết bị bao gồm ESP32, cảm biến AS608, LED và buzzer. AS608 thực hiện nhận dạng vân tay và giao tiếp với ESP32 thông qua UART. ESP32 tiếp nhận kết quả nhận dạng, xử lý nghiệp vụ chấm công và điều khiển các thiết bị phản hồi. Sau đó, ESP32 sử dụng Wi-Fi để trao đổi dữ liệu với Firebase.

Lớp dịch vụ được xây dựng trên nền tảng Firebase. Firebase Authentication chịu trách nhiệm xác thực tài khoản người dùng. Firebase Realtime Database lưu trữ thông tin nhân viên, ca làm việc và dữ liệu chấm công. Firebase Storage được sử dụng để lưu trữ dữ liệu hình ảnh đại diện.

Lớp ứng dụng là ứng dụng Smart Attendance được phát triển bằng React Native và TypeScript trên nền tảng Android. Đây là thành phần trọng tâm của đề tài xét trên góc độ phát triển ứng dụng di động. Ứng dụng cung cấp giao diện tương tác với dữ liệu Firebase và phân chia chức năng theo hai vai trò Admin và Employee.

Mô hình phân tầng giúp tách biệt nhiệm vụ giữa phần cứng, dịch vụ dữ liệu và ứng dụng. Nhờ đó, thay đổi trong giao diện ứng dụng không nhất thiết phải thay đổi trực tiếp cấu trúc xử lý của thiết bị, đồng thời dữ liệu có thể được sử dụng bởi nhiều thành phần khác nhau trong hệ thống.



Hình 3.2: Sơ đồ kiến trúc tổng thể hệ thống

3.1.4 Luồng dữ liệu tổng quát

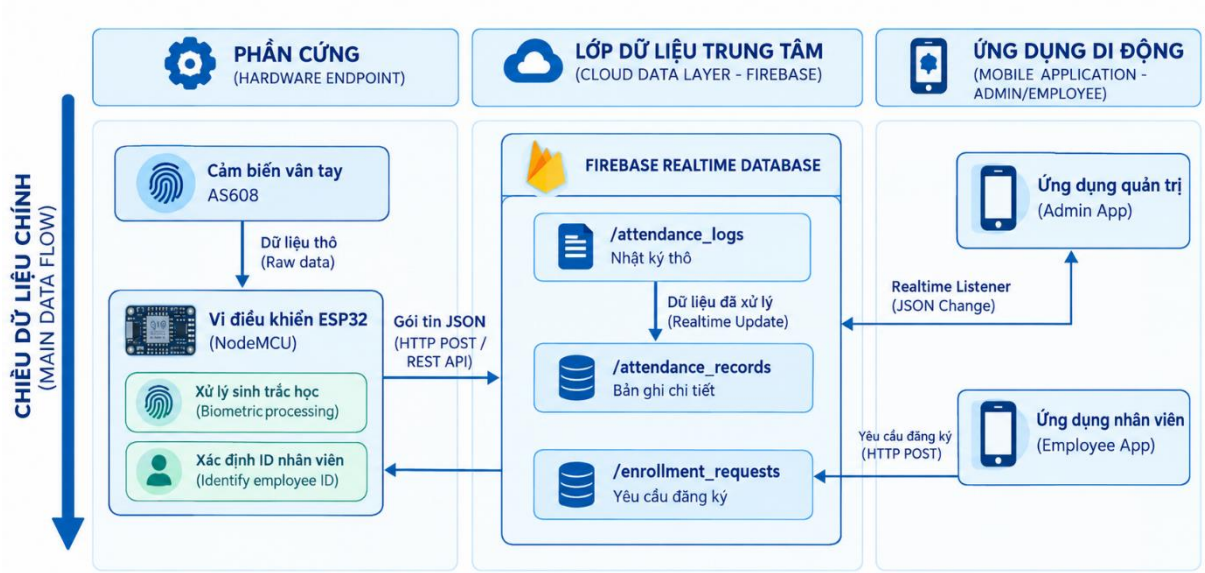
Luồng dữ liệu của hệ thống bắt đầu từ thao tác xác thực của người dùng tại cảm biến AS608. Sau khi nhận dạng thành công, cảm biến trả về mã định danh vân tay cho ESP32 thông qua UART.

ESP32 sử dụng mã định danh này để xác định nhân viên tương ứng. Hệ thống tiếp tục kiểm tra dữ liệu chấm công hiện tại và xác định thao tác Check-in hoặc Check-out. Kết quả xử lý được ghi nhận vào Firebase Realtime Database thông qua REST API.

Sau khi dữ liệu trên Firebase thay đổi, ứng dụng Smart Attendance nhận dữ liệu thông qua cơ chế Realtime Listener. Các màn hình đang theo dõi dữ liệu tương ứng sẽ cập nhật trạng thái giao diện.

Đối với quá trình quản lý nhân viên và đăng ký vân tay, luồng dữ liệu có chiều ngược lại. Quản trị viên thực hiện thao tác trên ứng dụng, dữ liệu được cập nhật lên Firebase và ESP32 có thể sử dụng dữ liệu yêu cầu để thực hiện thao tác tương ứng tại thiết bị.

Như vậy, Firebase đóng vai trò là lớp dữ liệu trung tâm kết nối giữa trạm chấm công và ứng dụng di động.



Hình 3.3: Sơ đồ luồng dữ liệu ESP32 → Firebase → Mobile App

3.2 Thiết kế phần cứng và thiết bị chấm công

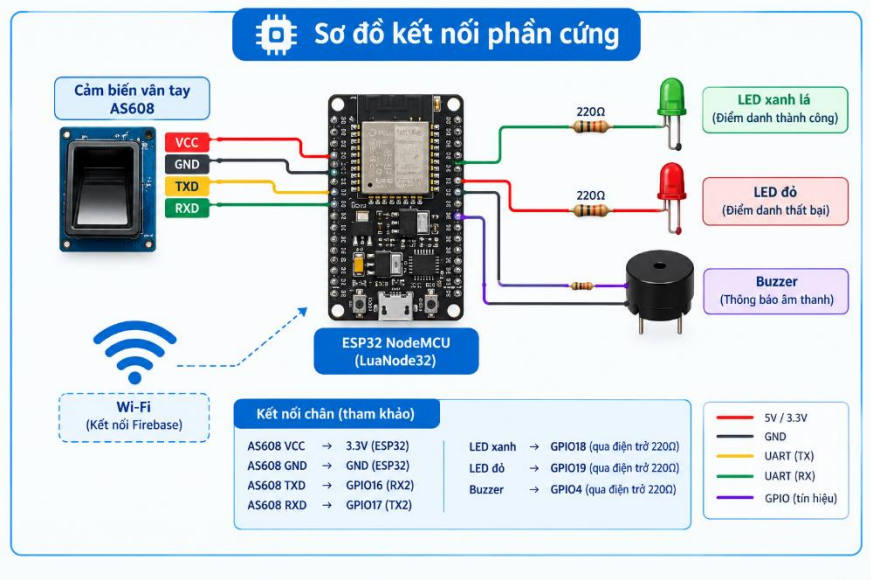
3.2.1 Thiết kế phần cứng ESP32

Thiết bị chấm công sử dụng ESP32 NodeMCU LuaNode32 làm bộ xử lý trung tâm. ESP32 được kết nối với cảm biến AS608 để nhận kết quả nhận dạng vân tay, đồng thời điều khiển LED xanh, LED đỏ và buzzer để phản hồi trực tiếp.

Trong sơ đồ, UART2 của ESP32 được sử dụng để giao tiếp với AS608. Chân GPIO16 được sử dụng làm RX và GPIO17 được sử dụng làm TX. Tốc độ truyền được cấu hình ở mức 57.600 bps.

LED xanh được kết nối với GPIO18, LED đỏ được kết nối với GPIO19 và buzzer được điều khiển thông qua GPIO4.

ESP32 sử dụng kết nối Wi-Fi để giao tiếp với Firebase. Sau khi nhận dạng vân tay và xử lý nghiệp vụ, thiết bị thực hiện các request HTTP cần thiết để đọc hoặc cập nhật dữ liệu.



Hình 3.4: Sơ đồ kết nối phần cứng

3.2.2 Thiết kế kết nối ESP32 và AS608

AS608 được kết nối với ESP32 thông qua giao tiếp UART. Hai đường truyền dữ liệu được kết nối chéo giữa hai thiết bị, trong đó TX của AS608 kết nối đến RX của ESP32 và RX của AS608 kết nối đến TX của ESP32.

Cấu hình kết nối được sử dụng trong hệ thống như sau:

Thành phần	ESP32
AS608 TX	GPIO16 – RX
AS608 RX	GPIO17 – TX
Tốc độ UART	57.600 bps

Thông qua kết nối này, ESP32 có thể gửi lệnh điều khiển đến AS608 và nhận kết quả nhận dạng từ cảm biến.

3.2.3 Thiết kế hệ thống chỉ báo

Hệ thống sử dụng hai LED và một buzzer để phản hồi trực tiếp sau quá trình xử lý.

Khi chấm công thành công, ESP32 điều khiển LED xanh và phát tín hiệu buzzer. Khi chấm công thất bại, LED đỏ được kích hoạt và buzzer cũng phát tín hiệu phản hồi.

Việc sử dụng các tín hiệu chỉ báo tại thiết bị giúp người dùng nhận biết kết quả ngay sau khi đặt ngón tay lên cảm biến mà không cần kiểm tra ứng dụng.

Trong trường hợp cảm biến xảy ra lỗi, hệ thống sử dụng trạng thái LED tương ứng để biểu thị tình trạng thiết bị theo logic được triển khai trong chương trình.

3.3 Thiết kế phần mềm ESP32

3.3.1 Kiến trúc phần mềm ESP32

Chương trình ESP32 được tổ chức theo hướng module hóa nhằm phân tách các chức năng điều khiển thiết bị, xử lý nghiệp vụ và giao tiếp dữ liệu.

Các module chính gồm AttendanceProcessor, AttendanceRepository, AttendanceService, EnrollmentRepository, EnrollmentService, FingerprintSensor, FirebaseREST, HrRepository, Indicator, NtpClock, WiFiService và main.

Trong đó, FingerprintSensor đảm nhiệm giao tiếp với AS608; WiFiService quản lý kết nối mạng; FirebaseREST thực hiện trao đổi dữ liệu với Firebase; các module Attendance xử lý nghiệp vụ chấm công; các module Enrollment phục vụ quá trình đăng ký vân tay; Indicator điều khiển LED và buzzer.

Cách tổ chức này giúp chương trình có sự phân tách giữa lớp giao tiếp phần cứng, lớp xử lý nghiệp vụ và lớp giao tiếp dữ liệu.

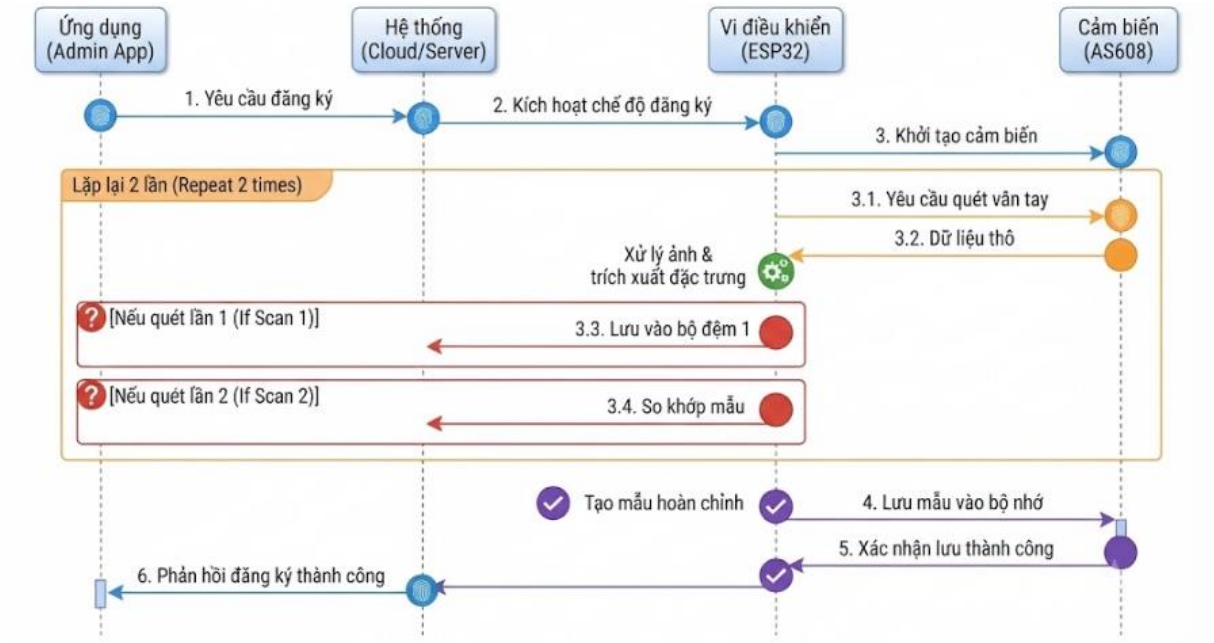
3.3.2 Thiết kế chức năng đăng ký vân tay

Quá trình đăng ký vân tay được thực hiện thông qua cơ chế yêu cầu từ ứng dụng và xử lý tại trạm phần cứng. Khi quản trị viên yêu cầu đăng ký vân tay cho một nhân viên, hệ thống tạo một yêu cầu đăng ký và gắn yêu cầu với nhân viên cùng thiết bị thực hiện.

AS608 thực hiện quá trình thu nhận vân tay theo hai lần quét. Lần quét đầu tiên được chuyển thành dữ liệu đặc trưng và lưu vào bộ đệm thứ nhất. Sau đó, người dùng phải nhấc ngón tay ra khỏi cảm biến trước khi thực hiện lần quét thứ hai.

Lần quét thứ hai được chuyển thành dữ liệu đặc trưng trong bộ đệm thứ hai. AS608 thực hiện đối chiếu hai bộ đệm để tạo mẫu vân tay hoàn chỉnh. Nếu quá trình ghép mẫu thành công, mẫu được lưu vào vị trí bộ nhớ được xác định trước.

Quá trình đăng ký gồm các bước chính: tìm vị trí trống, thu nhận lần quét thứ nhất, yêu cầu nhấc ngón tay, thu nhận lần quét thứ hai, tạo mẫu và lưu mẫu vào bộ nhớ cảm biến.



Hình 3.5: Sơ đồ tuần tự quá trình đăng ký vân tay

3.3.3 Thiết kế chức năng nhận dạng và xử lý chấm công

Khi người dùng đặt ngón tay lên cảm biến, hệ thống thực hiện quá trình tìm kiếm mẫu vân tay tương ứng trong bộ nhớ AS608. Để hạn chế trường hợp đọc không ổn định, chương trình thực hiện cơ chế kiểm chứng nhiều lần trước khi chấp nhận kết quả nhận

dạng.

Theo thiết kế hiện tại, quá trình đọc được kiểm chứng qua ba lần liên tiếp với khoảng cách 80 ms. Cơ chế này nhằm hạn chế các lượt đọc không ổn định do thao tác đặt ngón tay hoặc điều kiện thu nhận. Đây không phải là cơ chế phát hiện ngón tay giả hoặc chống giả mạo sinh trắc học.

Sau khi AS608 trả về mã định danh hợp lệ, ESP32 sử dụng mã này để xác định nhân viên tương ứng. Hệ thống sau đó xác định trạng thái chấm công.

Lượt quét đầu tiên trong ngày được xử lý là Check-in. Các lượt quét tiếp theo được xử lý theo trạng thái hiện tại của bản ghi chấm công. Check-out chỉ được phép thực hiện sau khi thời gian kể từ Check-in đạt điều kiện tối thiểu 5 phút theo thiết kế của hệ thống.

Sau khi xác định trạng thái, hệ thống ghi dữ liệu lên Firebase và cập nhật các thông tin liên quan đến thời gian chấm công.

3.3.4 Thiết kế đồng bộ thời gian

Thời gian đóng vai trò quan trọng trong hệ thống vì được sử dụng để xác định Check-in, Check-out, trạng thái đi trễ, thời gian làm việc và thời gian tăng ca.

ESP32 sử dụng cơ chế đồng bộ thời gian qua NTP để lấy thời gian từ mạng. Thời gian sau khi được đồng bộ được sử dụng làm cơ sở cho việc tạo timestamp của các sự kiện chấm công.

Việc đồng bộ thời gian giúp trạm chấm công có thể sử dụng một mốc thời gian thống nhất thay vì phụ thuộc vào thời gian trên thiết bị di động.

3.3.5 Thiết kế giao tiếp Firebase REST

ESP32 giao tiếp với Firebase thông qua REST API. Module FirebaseREST thực hiện việc tạo URL truy cập đến node dữ liệu và gửi HTTP request.

Phương thức GET được sử dụng để đọc dữ liệu từ Firebase, trong khi PATCH được sử dụng để cập nhật một phần dữ liệu. Dữ liệu được trao đổi dưới dạng JSON và request sử dụng application/json.

Thiết kế này cho phép ESP32 giao tiếp trực tiếp với Firebase mà không cần xây dựng thêm một máy chủ trung gian riêng cho thiết bị.

3.4 Thiết kế cơ sở dữ liệu và ứng dụng di động

3.4.1 Thiết kế cơ sở dữ liệu Firebase

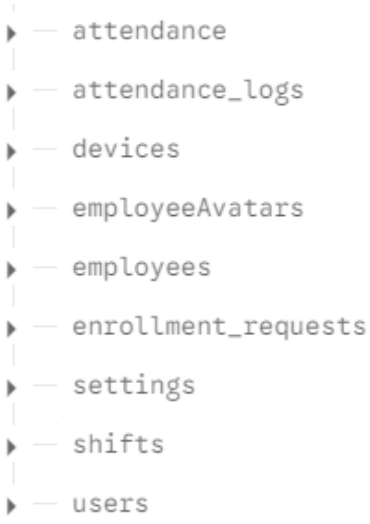
Firebase Realtime Database được sử dụng làm cơ sở dữ liệu trung tâm của hệ thống. Dữ liệu được tổ chức dưới dạng cây JSON và phân chia theo từng nhóm nghiệp vụ.

Node	Dữ liệu chính	Chức năng
/users/{uid}	email, role, active, employeeId	Quản lý tài khoản và vai trò
/employees/{employeeId}	name, department, fingerprintId, assignedShiftId, active	Quản lý nhân viên
/shifts/{shiftId}	startTime, endTime, gracePeriodMinutes, active	Quản lý ca làm việc
/attendance/{date}/{employeeId}	checkIn, checkOut, status, lateMinutes, workingHours, overtimeHours	Lưu bản ghi chấm công
/attendance_logs/{logId}	fingerprintId, employeeId, outcome, timestamp	Lưu sự kiện quét
/enrollment_requests/{requestId}	employeeId, deviceId, status, fingerprintId, step	Quản lý yêu cầu đăng ký
/devices/{deviceId}	status, lastSeen, capacity, storedTemplates	Quản lý thiết bị
/settings	companyName, graceMinutesGlobal	Lưu cấu hình hệ thống

Cách tổ chức dữ liệu trên cho phép mỗi nhóm chức năng có một phạm vi dữ liệu

tương đối độc lập. Đặc biệt, việc tách attendance và attendance_logs giúp hệ thống vừa lưu bản ghi tổng hợp của một ngày làm việc, vừa lưu được lịch sử của từng lượt quét.

Mối liên hệ giữa fingerprintId trong hồ sơ nhân viên và mã định danh do AS608 trả về là cơ sở để liên kết dữ liệu nhận dạng tại thiết bị với thông tin nhân viên trên hệ thống.



Hình 3.6: Sơ đồ cấu trúc Firebase Realtime Database

3.4.2 Thiết kế xác thực và phân quyền

Ứng dụng sử dụng Firebase Authentication để thực hiện đăng nhập bằng Email và Password. Sau khi xác thực thành công, ứng dụng truy vấn thông tin người dùng tại /users/{uid} để xác định vai trò và thông tin liên kết với hồ sơ nhân viên.

Hai vai trò chính được sử dụng là Admin và Employee.

Admin có quyền truy cập các chức năng quản trị như quản lý nhân viên, đăng ký vân tay, quản lý ca làm việc, lịch sử chấm công và báo cáo.

Employee được giới hạn trong các chức năng liên quan đến thông tin cá nhân, ca làm việc và lịch sử chấm công của bản thân.

Việc phân quyền được thể hiện ở tầng giao diện thông qua điều hướng theo vai trò. Tuy nhiên, thiết kế hệ thống cũng xác định rằng việc ẩn hoặc hiện giao diện không thể thay thế cơ chế bảo vệ dữ liệu tại Firebase. Firebase Security Rules cần được cấu hình để kiểm soát quyền đọc và ghi dữ liệu theo vai trò thực tế.

3.4.3 Thiết kế cấu trúc ứng dụng React Native

Ứng dụng Smart Attendance được tổ chức theo cấu trúc module nhằm tách biệt

giao diện, dịch vụ dữ liệu và các kiểu dữ liệu.

Các nhóm thành phần chính gồm:

components: chứa các thành phần giao diện có khả năng tái sử dụng.

screens: chứa các màn hình chính của ứng dụng như đăng nhập, Dashboard, nhân viên, ca làm việc, lịch sử và báo cáo.

services: chứa các module giao tiếp với Firebase và xử lý các dịch vụ của ứng dụng.

types: định nghĩa các kiểu dữ liệu được sử dụng trong ứng dụng.

utils: chứa các hàm tiện ích phục vụ xử lý dữ liệu và nghiệp vụ.

App.tsx đóng vai trò điều phối luồng khởi động và điều hướng chính của ứng dụng dựa trên trạng thái xác thực và vai trò người dùng.

Cấu trúc này giúp phần giao diện không phải trực tiếp xử lý toàn bộ logic truy cập dữ liệu, từ đó tạo sự phân tách rõ ràng giữa giao diện và tầng dịch vụ.

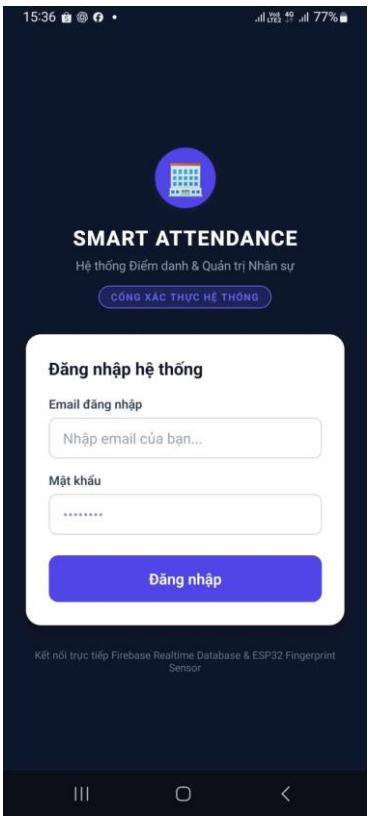
3.4.4 Thiết kế giao diện đăng nhập

Màn hình đăng nhập là điểm bắt đầu của ứng dụng Smart Attendance. Người dùng nhập Email và Password để thực hiện xác thực thông qua Firebase Authentication.

Trong quá trình đăng nhập, ứng dụng cần xử lý các trạng thái như đang xác thực, thông tin đăng nhập không hợp lệ, tài khoản không tồn tại và tài khoản bị vô hiệu hóa.

Sau khi xác thực thành công, ứng dụng lấy thông tin vai trò từ Firebase và điều hướng người dùng đến giao diện tương ứng.

Thiết kế màn hình đăng nhập tập trung vào hai yếu tố là thao tác đơn giản và phản hồi rõ ràng khi xảy ra lỗi.



Hình 3.7: Giao diện Login

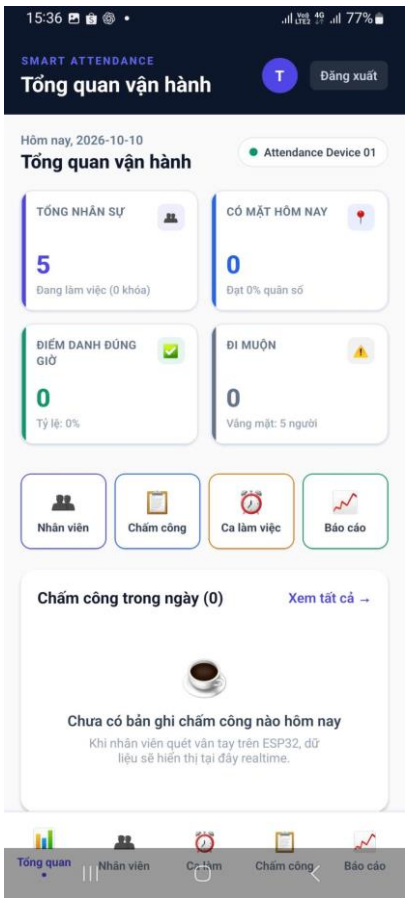
3.4.5 Thiết kế Dashboard Admin

Dashboard Admin là màn hình tổng quan dành cho quản trị viên. Màn hình cung cấp thông tin tổng hợp về tình trạng nhân sự và hoạt động chấm công.

Dashboard bao gồm trạng thái kết nối của thiết bị ESP32, các thẻ thống kê nhanh và khu vực hiển thị những hoạt động chấm công gần nhất.

Các nhóm chức năng chính được tổ chức để quản trị viên có thể nhanh chóng truy cập quản lý nhân viên, quản lý ca làm việc và đăng ký vân tay.

Dữ liệu Dashboard được lấy từ Firebase và cập nhật thông qua cơ chế Realtime Listener. Khi dữ liệu chấm công thay đổi, giao diện có thể cập nhật mà không yêu cầu người dùng tải lại toàn bộ màn hình.



Hình 3.8: Giao diện Dashboard Admin

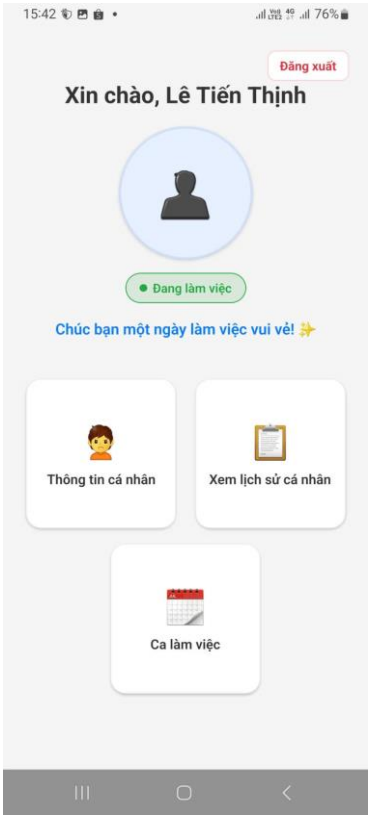
3.4.6 Thiết kế Dashboard Employee

Dashboard Employee được thiết kế dành cho người dùng thuộc vai trò nhân viên. Màn hình hiển thị thông tin cá nhân, trạng thái làm việc trong ngày và ca làm việc được phân công.

Thông tin trạng thái được xây dựng dựa trên dữ liệu chấm công hiện tại. Người dùng có thể theo dõi thời gian Check-in, Check-out và các thông tin liên quan đến trạng thái làm việc.

Khi có dữ liệu chấm công mới được ghi nhận trên Firebase trong lúc ứng dụng đang hoạt động, Dashboard có thể nhận thay đổi thông qua Realtime Listener và cập nhật giao diện tương ứng.

Kết quả chấm công có thể được hiển thị dưới dạng thông báo trong giao diện ứng dụng. Cơ chế này là cập nhật dữ liệu trong ứng dụng và không đồng nghĩa với Push Notification khi ứng dụng chạy nền hoặc đã đóng.



Hình 3.9: Giao diện Dashboard Employee

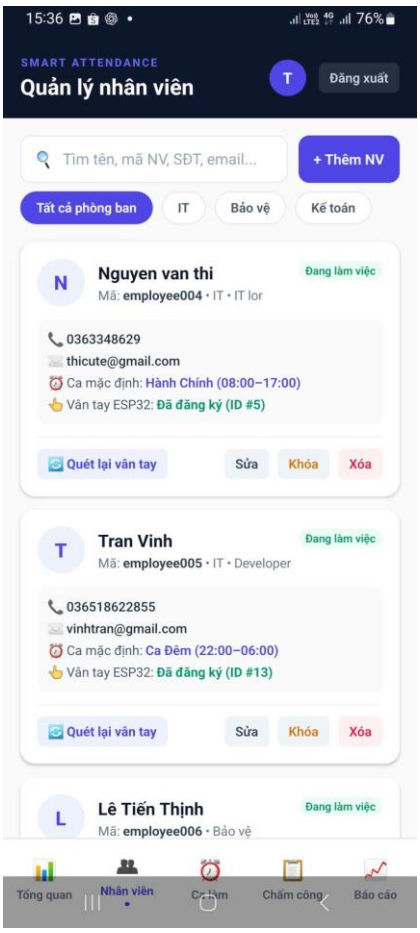
3.4.7 Thiết kế chức năng quản lý nhân viên

Màn hình quản lý nhân viên cho phép Admin xem danh sách nhân viên và thực hiện các thao tác quản lý hồ sơ.

Danh sách nhân viên hiển thị những thông tin cần thiết như họ tên, mã nhân viên, phòng ban, trạng thái hoạt động và tình trạng liên kết vân tay.

Ứng dụng cung cấp chức năng tìm kiếm để hỗ trợ quản trị viên nhanh chóng xác định nhân viên cần thao tác. Dữ liệu cũng có thể được lọc theo các thuộc tính phù hợp với cấu trúc thông tin nhân viên.

Khi thêm hoặc chỉnh sửa nhân viên, ứng dụng cập nhật dữ liệu tương ứng trên Firebase. Khi xóa nhân viên, hệ thống đồng thời cần xử lý mối liên kết giữa hồ sơ nhân viên và mẫu vân tay đã được lưu tại thiết bị.



Hình 3.10: Màn hình quản lí danh sách nhân viên

3.4.8 Thiết kế chức năng đăng ký vân tay

Chức năng đăng ký vân tay được khởi tạo từ ứng dụng Admin nhưng quá trình thu nhận mẫu thực tế được thực hiện tại thiết bị ESP32 và AS608.

Khi Admin chọn nhân viên và thực hiện yêu cầu đăng ký, ứng dụng tạo một bản ghi yêu cầu trong Firebase. Yêu cầu này chứa thông tin nhân viên, thiết bị và trạng thái xử lý.

ESP32 theo dõi các yêu cầu đăng ký và thực hiện quá trình thu nhận vân tay tại trạm. Sau khi hoàn tất, mã định danh vân tay được liên kết với hồ sơ nhân viên trên Firebase.

Quy trình này tạo ra sự liên kết giữa thao tác quản lý trên ứng dụng và quá trình thu nhận sinh trắc học tại thiết bị.

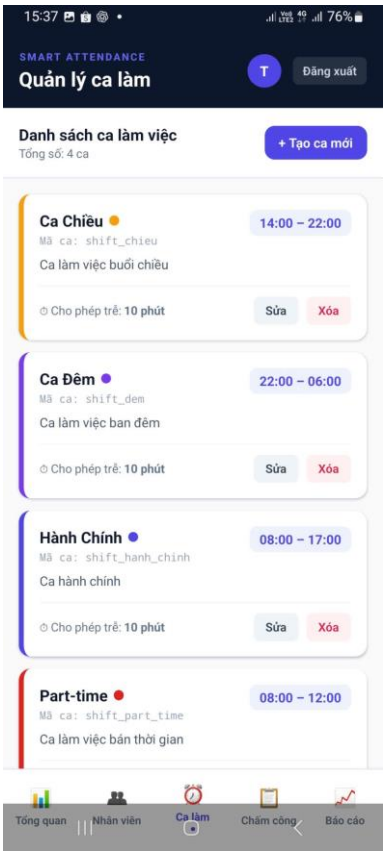
3.4.9 Thiết kế quản lý ca làm việc

Chức năng quản lý ca làm việc cho phép Admin tạo và chỉnh sửa các thông tin

liên quan đến ca. Dữ liệu ca được lưu trữ tại /shifts/{shiftId}.

Một ca làm việc bao gồm thời gian bắt đầu, thời gian kết thúc, thời gian ân hạn và trạng thái hoạt động.

Thông tin ca được liên kết với hồ sơ nhân viên thông qua assignedShiftId. Khi xử lý chấm công, hệ thống sử dụng thông tin ca để xác định trạng thái đúng giờ hoặc đi muộn, đồng thời phục vụ việc tính toán thời gian làm việc và tăng ca.



Hình 3.11: Màn hình quản lí ca làm việc

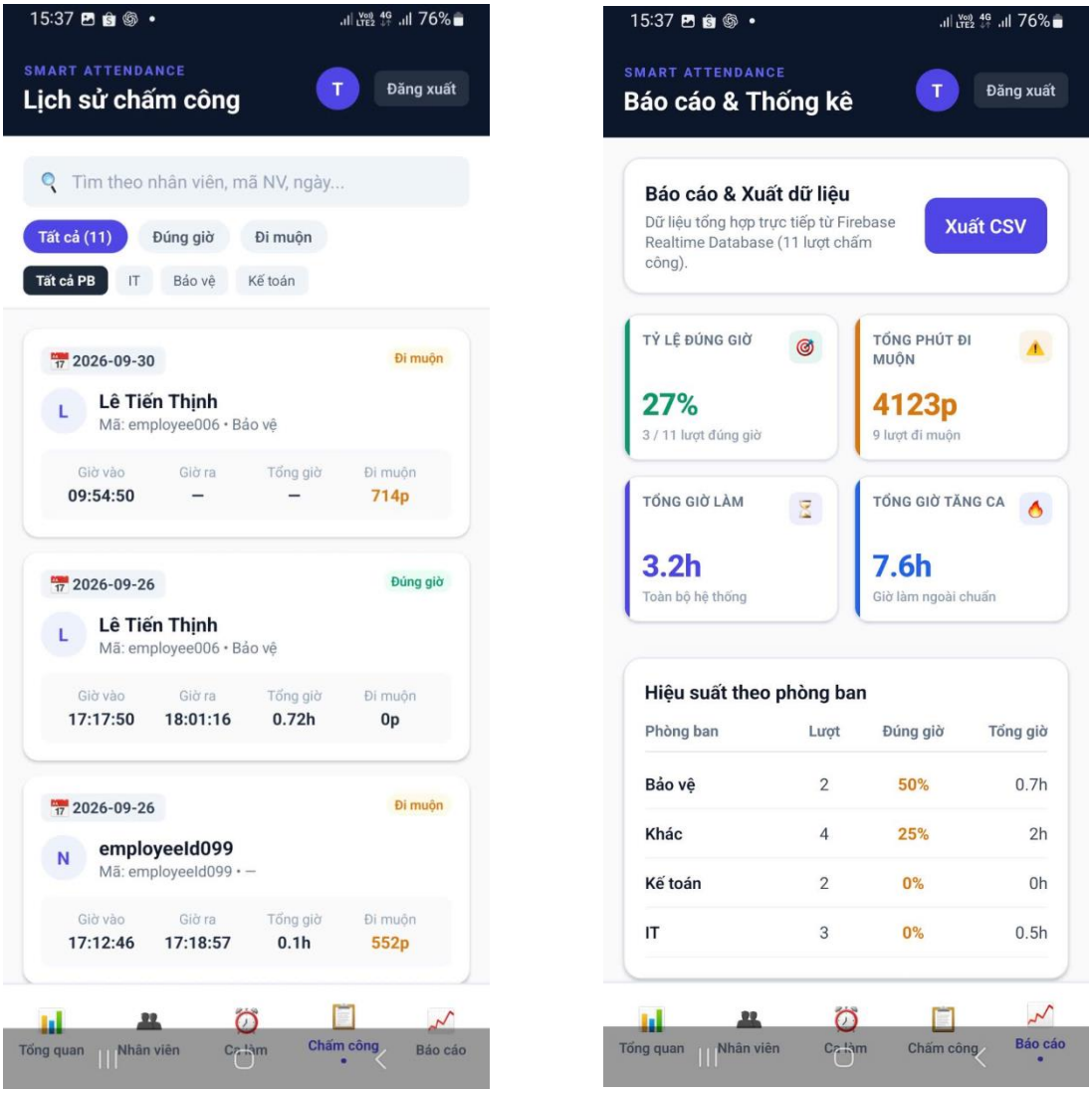
3.4.10 Thiết kế lịch sử chấm công và báo cáo

Màn hình lịch sử chấm công cho phép người dùng tra cứu các bản ghi đã được lưu trên Firebase. Đối với Admin, dữ liệu có thể được tìm kiếm và lọc để phục vụ việc quản lý nhân sự. Đối với Employee, dữ liệu được sử dụng để theo dõi lịch sử cá nhân.

Dữ liệu chấm công bao gồm thời gian Check-in, Check-out, trạng thái, số phút đi trễ, số giờ làm việc và số giờ tăng ca.

Trang báo cáo cung cấp các chỉ số tổng hợp phục vụ việc theo dõi tình hình chấm công. Các nhóm thống kê hiện tại bao gồm tỷ lệ đúng giờ, tổng số phút đi trễ, tổng số giờ làm việc và tổng số giờ tăng ca.

Đối với các màn hình chứa lượng dữ liệu lớn, việc giới hạn phạm vi dữ liệu theo ngày hoặc khoảng thời gian là cần thiết để tránh tải toàn bộ lịch sử trong mỗi lần mở màn hình.



Hình 3.12: Giao diện lịch sử chấm công và giao diện báo cáo

3.4.11 Thiết kế cơ chế đồng bộ dữ liệu trong ứng dụng

Ứng dụng sử dụng Firebase Realtime Database Listener để theo dõi những node dữ liệu cần thiết. Các màn hình Dashboard và lịch sử chấm công có thể đăng ký listener thông qua onValue(ref(...), callback).

Khi ESP32 cập nhật một bản ghi chấm công lên Firebase, listener nhận được sự thay đổi và cập nhật trạng thái của ứng dụng. React Native sau đó render lại giao diện dựa trên state mới.

Listener được quản lý theo vòng đời của màn hình nhằm hạn chế việc duy trì

những kết nối không còn cần thiết. Khi màn hình không còn sử dụng dữ liệu, listener cần được hủy đăng ký.

Ngoài ra, hệ thống còn có cơ chế syncDeletedFingerprints để đối chiếu dữ liệu nhân viên trên Firebase với các mẫu đang tồn tại trên AS608. Khi một nhân viên bị xóa và mẫu vân tay không còn liên kết với hồ sơ hợp lệ, hệ thống có thể xác định mẫu tương ứng để xử lý tại thiết bị.

3.5 Thiết kế luồng nghiệp vụ chính

3.5.1 Luồng đăng nhập

Quá trình đăng nhập bắt đầu khi người dùng nhập Email và Password trên ứng dụng. Thông tin được gửi đến Firebase Authentication để xác thực.

Nếu xác thực thất bại, ứng dụng hiển thị thông báo lỗi tương ứng. Nếu xác thực thành công, ứng dụng lấy UID của tài khoản và truy vấn thông tin tại /users/{uid}.

Dựa trên trường role, hệ thống xác định người dùng thuộc nhóm Admin hoặc Employee và điều hướng đến giao diện tương ứng.

3.5.2 Luồng Check-in

Nhân viên đặt ngón tay lên cảm biến AS608. Sau khi xác thực thành công, AS608 trả về fingerprintId cho ESP32.

ESP32 xác định nhân viên tương ứng và kiểm tra bản ghi chấm công trong ngày. Nếu chưa có Check-in, hệ thống ghi nhận thời gian hiện tại là thời gian Check-in.

Dựa trên ca làm việc được phân công và thời gian ân hạn, hệ thống xác định trạng thái chấm công. Dữ liệu sau đó được ghi vào Firebase.

Ứng dụng nhận dữ liệu mới thông qua Realtime Listener và cập nhật trạng thái trên giao diện.

3.5.3 Luồng Check-out

Khi nhân viên thực hiện quét vân tay lần tiếp theo, ESP32 xác định nhân viên và kiểm tra bản ghi Check-in hiện tại.

Nếu thời gian từ Check-in đến lượt quét đạt tối thiểu 5 phút, hệ thống cho phép ghi nhận Check-out. Thời gian làm việc được tính dựa trên khoảng thời gian giữa Check-in và Check-out.

Nếu điều kiện Check-out chưa được đáp ứng, hệ thống không ghi nhận lượt quét thành Check-out và trả về trạng thái phù hợp.

3.5.4 Luồng đăng ký vân tay

Admin lựa chọn nhân viên cần đăng ký vân tay trên ứng dụng và tạo yêu cầu đăng ký.

Yêu cầu được ghi vào Firebase với thông tin nhân viên, thiết bị và trạng thái xử lý. ESP32 nhận yêu cầu và chuyển sang trạng thái xử lý.

Người dùng đặt cùng một ngón tay lên AS608 theo hai lần quét. Cảm biến tạo mẫu từ hai lần thu nhận và lưu mẫu vào vị trí bộ nhớ được xác định.

Sau khi hoàn thành, mã vân tay được liên kết với hồ sơ nhân viên trên Firebase. Ứng dụng nhận trạng thái mới của yêu cầu và hiển thị kết quả cho Admin.

3.5.5 Luồng xóa nhân viên

Khi Admin xóa một nhân viên, thông tin hồ sơ tương ứng được xóa hoặc vô hiệu hóa trên Firebase theo logic của hệ thống.

Sau đó, hệ thống cần kiểm tra mối liên hệ giữa nhân viên và mẫu vân tay đã được lưu trên AS608. Nếu mẫu không còn thuộc về một nhân viên hợp lệ, hệ thống thực hiện cơ chế đồng bộ để tránh tồn tại mẫu vân tay không còn liên kết với hồ sơ nhân viên.

3.6 Thiết kế điều hướng ứng dụng

Ứng dụng sử dụng cơ chế điều hướng dựa trên trạng thái xác thực và vai trò của người dùng.

Khi ứng dụng được khởi động, hệ thống kiểm tra trạng thái đăng nhập hiện tại. Nếu chưa xác thực, người dùng được đưa đến màn hình Login. Nếu đã xác thực, ứng dụng lấy thông tin tài khoản và vai trò để xác định nhóm giao diện cần hiển thị.

Đối với Admin, hệ thống cung cấp các khu vực chính gồm Dashboard, Employees, Shifts, History và Reports.

Đối với Employee, giao diện được giới hạn ở các chức năng liên quan đến thông tin cá nhân, ca làm việc, trạng thái hiện tại và lịch sử chấm công.

Cách tổ chức điều hướng theo vai trò giúp giảm sự phức tạp của giao diện và đảm bảo người dùng tập trung vào các chức năng phù hợp với nhiệm vụ của mình.

CHƯƠNG 4: XÂY DỰNG, TÍCH HỢP VÀ KIỂM THỬ HỆ THỐNG

4.1 Xây dựng hệ thống phần cứng

4.1.1 Lắp ráp ESP32 và AS608

Quá trình xây dựng trạm chấm công được thực hiện trên nền tảng vi điều khiển ESP32 NodeMCU LuaNode32 kết hợp với cảm biến vân tay AS608. Các thành phần phần cứng được lắp ráp trên bo mạch thử nghiệm và bố trí trong hộp mô hình nhằm tạo thành một thiết bị chấm công có khả năng thao tác trực tiếp với người sử dụng.

Cảm biến AS608 được kết nối với ESP32 thông qua giao tiếp UART. Trong thiết kế thực tế, chân TX của AS608 được kết nối với chân GPIO 16, tương ứng với RX2 của ESP32; chân RX của AS608 được kết nối với GPIO 17, tương ứng với TX2. Chân GND của cảm biến được nối chung với GND của ESP32. Giao tiếp UART giữa hai thiết bị được cấu hình ở tốc độ 57.600 bps.

Việc bố trí cảm biến được thực hiện sao cho bề mặt quét vân tay nằm ở vị trí thuận tiện để người dùng đặt ngón tay. Khi cấp nguồn, chương trình trên ESP32 thực hiện kiểm tra khả năng giao tiếp với AS608 trước khi cho phép thực hiện các thao tác đăng ký hoặc xác thực vân tay.

Trong quá trình lắp ráp, việc kiểm tra nguồn cấp và đường truyền UART được thực hiện trước nhằm hạn chế các lỗi do kết nối phần cứng. Đối với các thành phần sử dụng mức điện áp hoặc dòng điện vượt quá khả năng trực tiếp của GPIO, cần sử dụng mạch điều khiển hoặc mạch đệm phù hợp với thông số thực tế của linh kiện.

4.1.2 Kết nối LED và buzzer

Hệ thống sử dụng hai LED để biểu thị trạng thái hoạt động của trạm chấm công. LED xanh được kết nối với GPIO 18 và LED đỏ được kết nối với GPIO 19. Các LED được bố trí ở mặt trước của thiết bị để người dùng có thể quan sát trạng thái sau khi thực hiện thao tác quét vân tay.

Còi Buzzer được kết nối với GPIO 4 và được sử dụng để phát tín hiệu âm thanh tương ứng với từng trạng thái xử lý. Trong quá trình hoạt động, LED và Buzzer phối hợp với nhau để tạo phản hồi tại chỗ, giúp người dùng nhận biết kết quả mà không cần quan sát ứng dụng di động.

Các trạng thái phản hồi của thiết bị được triển khai như sau. Khi hệ thống ở trạng

thái chờ, các LED và Buzzer không phát tín hiệu. Khi cảm biến đọc được vân tay, LED xanh được bật và Buzzer phát một tiếng bíp ngắn để xác nhận quá trình thu nhận. Khi chấm công hoặc đăng ký vân tay thành công, LED xanh được bật trong khoảng thời gian xác định và Buzzer phát hai tiếng bíp ngắn. Ngược lại, khi vân tay không khớp hoặc lượt quét bị từ chối, LED đỏ được bật và Buzzer phát một tiếng bíp dài. Trong trường hợp cảm biến xảy ra lỗi, LED đỏ được duy trì để biểu thị trạng thái lỗi và Buzzer không phát tín hiệu.

Bảng 4.1: Trạng thái LED và Buzzer của thiết bị

Trạng thái hệ thống / Sự kiện	**LED Xanh**	**LED Đỏ**	**Còi Buzzer**	**Ý nghĩa**
Trạng thái chờ	Tắt	Tắt	Không phát tín hiệu	Thiết bị sẵn sàng, đang chờ người dùng quét vân tay.
Chấm công / Đăng ký thành công	Bật	Tắt	2 tiếng bíp ngắn	Giao dịch điểm danh hoặc đăng ký vân tay hoàn tất, dữ liệu chuẩn bị gửi lên Firebase.
Vân tay không khớp / Lỗi quét	Tắt	Bật	1 tiếng bíp dài	Vân tay không tồn tại trong bộ nhớ hoặc góc đặt tay không đúng, yêu cầu quét lại.
Lỗi cảm biến / Kết nối phần cứng	Tắt	Duy trì sáng	Không phát tín hiệu	Cảnh báo lỗi thiết bị ngoại vi hoặc mất kết nối với module AS608.

4.1.3 Lập trình ESP32

Phần mềm điều khiển ESP32 được xây dựng bằng C++ trên nền tảng Arduino Core for ESP32. Mã nguồn được tổ chức theo hướng module hóa nhằm tách biệt các

chức năng phần cứng, kết nối mạng, đồng bộ thời gian, giao tiếp Firebase và xử lý nghiệp vụ chấm công.

Module Indicator chịu trách nhiệm điều khiển LED và Buzzer, bao gồm các trạng thái sẵn sàng, đang xử lý, thành công, thất bại và lỗi cảm biến. Việc đóng gói các thao tác điều khiển chỉ thị giúp giảm sự phụ thuộc giữa phần xử lý nghiệp vụ và phần phản hồi phần cứng.

Module WiFiService đảm nhiệm việc kết nối ESP32 với mạng Wi-Fi và thực hiện cơ chế kết nối lại khi xảy ra gián đoạn. Module NtpClock sử dụng cơ chế đồng bộ thời gian mạng NTP để cung cấp thời gian phục vụ việc ghi nhận chấm công.

Module FirebaseREST đóng gói quá trình giao tiếp giữa ESP32 và Firebase Realtime Database thông qua giao thức HTTPS REST. Các phương thức GET, PUT, POST, PATCH và DELETE được sử dụng tùy theo loại thao tác dữ liệu.

Ngoài ra, các module liên quan đến cảm biến vân tay và nghiệp vụ được tổ chức riêng, trong đó FingerprintSensor quản lý giao tiếp với AS608; EnrollmentService xử lý đăng ký vân tay; AttendanceService xử lý chấm công; và các chức năng liên quan đến xóa mẫu vân tay được sử dụng để đồng bộ dữ liệu giữa cảm biến và hồ sơ nhân viên trên Firebase.

Cách tổ chức này giúp mã nguồn ESP32 có cấu trúc rõ ràng hơn, đồng thời tạo điều kiện thuận lợi cho việc kiểm thử và bảo trì từng thành phần độc lập.

4.1.4 Xây dựng chức năng nhận dạng và chấm công

Chức năng nhận dạng vân tay được triển khai dựa trên khả năng lưu trữ và đối sánh mẫu của cảm biến AS608. Khi đăng ký vân tay, hệ thống thực hiện quá trình thu nhận mẫu hai lần. Sau lần quét thứ nhất, người dùng được yêu cầu nhấc ngón tay khỏi cảm biến trước khi đặt lại cùng ngón tay để thực hiện lần quét thứ hai. Hai mẫu thu được được AS608 đối sánh và tạo thành mẫu vân tay lưu trữ trong một vị trí trống của bộ nhớ cảm biến.

Đối với quá trình nhận dạng, hệ thống thực hiện cơ chế kiểm tra nhiều lần nhằm hạn chế các trường hợp ảnh vân tay không ổn định. Cảm biến được yêu cầu thu nhận ảnh qua ba lần đọc liên tiếp, với khoảng cách giữa các lần đọc là 80 ms. Khi kết quả đủ ổn định, AS608 thực hiện đối sánh với các mẫu đã lưu và trả về mã vân tay tương ứng nếu xác thực thành công.

Sau khi nhận được mã vân tay, ESP32 thực hiện tra cứu thông tin nhân viên trên Firebase và xử lý nghiệp vụ chấm công. Lượt quét đầu tiên trong ngày được xác định là Check-in, trong khi lượt quét tiếp theo được sử dụng để xác định Check-out theo trạng thái hiện tại của bản ghi.

Hệ thống cũng thực hiện tính toán trạng thái đúng giờ hoặc đi muộn dựa trên thời gian bắt đầu ca và khoảng thời gian ân hạn được cấu hình. Khi có thời gian Check-out, hệ thống tính toán tổng thời gian làm việc và thời gian tăng ca dựa trên thời gian kết thúc ca.

Để hạn chế việc một người vô tình quét nhiều lần trong thời gian ngắn, cơ chế chống quét lặp được triển khai với khoảng thời gian 300 giây. Nếu cùng một lượt quét được phát hiện trong khoảng thời gian này, hệ thống bỏ qua lượt quét lặp và không ghi đè thời gian chấm công trước đó.

Bên cạnh đó, ESP32 định kỳ kiểm tra các yêu cầu đăng ký vân tay được gửi từ ứng dụng thông qua Firebase. Khi nhận được yêu cầu hợp lệ, thiết bị thực hiện quy trình đăng ký vân tay và cập nhật trạng thái xử lý trở lại cơ sở dữ liệu.

4.2 Xây dựng ứng dụng di động và cơ sở dữ liệu

4.2.1 Xây dựng giao diện ứng dụng

Ứng dụng di động Smart Attendance được xây dựng bằng React Native 0.87.1 kết hợp TypeScript. Giao diện được thiết kế theo hướng phù hợp với thiết bị Android và được tổ chức thành các màn hình tương ứng với vai trò của người dùng.

Ứng dụng sử dụng một bộ cài duy nhất cho hai nhóm người dùng là Admin và Employee. Sau khi đăng nhập, hệ thống xác định vai trò của tài khoản và điều hướng người dùng đến phân hệ tương ứng.

Đối với Admin, thanh điều hướng chính gồm các chức năng Dashboard, Employees, Shifts, History và Reports. Dashboard cung cấp thông tin tổng quan về hệ thống; Employees hỗ trợ quản lý nhân viên và đăng ký vân tay; Shifts quản lý ca làm việc; History cung cấp lịch sử chấm công; Reports hỗ trợ tổng hợp các thông tin thống kê liên quan đến chấm công.

Đối với Employee, giao diện tập trung vào thông tin cá nhân, ca làm việc, trạng thái chấm công trong ngày và lịch sử làm việc. Sau khi hệ thống nhận được kết quả

chấm công từ thiết bị, ứng dụng có thể cập nhật trạng thái và hiển thị kết quả trong giao diện khi ứng dụng đang hoạt động.

Một số thành phần giao diện dùng chung được xây dựng nhằm duy trì tính nhất quán giữa các màn hình. Các trạng thái Loading, Empty và Error cũng được xử lý để giúp người dùng nhận biết trạng thái dữ liệu trong quá trình ứng dụng giao tiếp với Firebase.

4.2.2 Xây dựng các chức năng của ứng dụng

Chức năng đăng nhập được triển khai thông qua Firebase Authentication với phương thức Email và Password. Khi người dùng đăng nhập thành công, ứng dụng truy vấn thông tin tương ứng tại /users/{uid} để xác định vai trò và trạng thái tài khoản.

Nếu tài khoản thuộc vai trò Admin, ứng dụng điều hướng đến giao diện quản trị. Nếu tài khoản thuộc vai trò Employee, ứng dụng điều hướng đến giao diện dành cho nhân viên. Trường hợp tài khoản bị vô hiệu hóa, hệ thống thực hiện đăng xuất và hiển thị thông báo tương ứng.

Đối với phân hệ quản trị nhân viên, ứng dụng cung cấp các chức năng thêm, chỉnh sửa và xóa nhân viên. Khi tạo tài khoản nhân viên, hệ thống đồng thời xây dựng hồ sơ nhân viên tương ứng trên Firebase. Đối với thao tác xóa, hệ thống cần đảm bảo dữ liệu hồ sơ được xử lý đồng bộ với thông tin tài khoản và mẫu vân tay liên quan.

Chức năng quản lý ca làm việc cho phép Admin tạo và chỉnh sửa thông tin ca, bao gồm thời gian bắt đầu, thời gian kết thúc và thời gian ân hạn. Hệ thống cũng hỗ trợ xử lý các trường hợp ca làm việc kéo dài qua ngày hôm sau.

Chức năng đăng ký vân tay được thực hiện theo cơ chế gửi yêu cầu từ ứng dụng đến Firebase. Admin lựa chọn nhân viên cần đăng ký, sau đó hệ thống tạo một yêu cầu đăng ký trên Firebase. ESP32 kiểm tra yêu cầu, thực hiện quá trình thu nhận vân tay tại cảm biến AS608 và cập nhật kết quả cùng mã vân tay vào Firebase.

Đối với chức năng lịch sử và báo cáo, ứng dụng truy xuất dữ liệu chấm công từ Firebase Realtime Database. Các thông tin như thời gian Check-in, Check-out, số phút đi muộn, tổng thời gian làm việc và thời gian tăng ca được sử dụng để hiển thị cho người dùng và phục vụ thống kê.

4.2.3 Xây dựng cơ sở dữ liệu Firebase

Firebase Realtime Database được sử dụng làm cơ sở dữ liệu trung tâm của hệ thống. Cơ sở dữ liệu được khởi tạo tại khu vực asia-southeast1 và tổ chức theo cấu trúc JSON, trong đó mỗi node đảm nhiệm một nhóm nghiệp vụ riêng.

Các nhóm dữ liệu chính bao gồm /users, /employees, /shifts, /attendance, /attendance_logs, /enrollment_requests, /devices và /settings.

Node /users/{uid} lưu thông tin liên quan đến tài khoản đăng nhập, bao gồm email, role, trạng thái active và mã nhân viên liên kết. Node /employees/{employeeId} lưu hồ sơ nhân viên, mã vân tay và ca làm việc được phân công.

Node /shifts/{shiftId} lưu thông tin ca làm việc, thời gian bắt đầu, thời gian kết thúc và khoảng thời gian ân hạn. Node /attendance/{date}/{employeeId} lưu bản ghi chấm công tổng hợp, bao gồm Check-in, Check-out, trạng thái, số phút đi muộn, thời gian làm việc và thời gian tăng ca.

Bên cạnh đó, /attendance_logs/{logId} lưu các sự kiện phát sinh từ từng lượt quét; /enrollment_requests/{requestId} quản lý các yêu cầu đăng ký vân tay; /devices/{deviceId} lưu trạng thái của thiết bị; và /settings lưu các cấu hình chung của hệ thống.

Trong phiên bản thử nghiệm, quy tắc truy cập được thiết lập ở mức xác thực người dùng. Tuy nhiên, cơ chế này chưa thể hiện đầy đủ phân quyền theo vai trò và phạm vi dữ liệu. Vì vậy, trước khi triển khai trên dữ liệu thực tế, Firebase Security Rules cần được hoàn thiện để giới hạn quyền đọc và ghi phù hợp với Admin, Employee và thiết bị.

4.2.4 Kết nối và đồng bộ ứng dụng với Firebase

Ứng dụng sử dụng cơ chế lắng nghe dữ liệu theo thời gian thực của Firebase Realtime Database. Tại các màn hình cần cập nhật liên tục như Dashboard và lịch sử chấm công, ứng dụng đăng ký listener để theo dõi sự thay đổi của dữ liệu.

Khi ESP32 ghi một bản ghi chấm công lên Firebase, listener tương ứng trên ứng dụng nhận được thay đổi và cập nhật trạng thái giao diện mà không yêu cầu người dùng tải lại thủ công. Cơ chế này tạo thành luồng dữ liệu từ thiết bị đến cơ sở dữ liệu và sau đó đến ứng dụng di động.

Trong quá trình xây dựng, các listener được gỡ bỏ khi màn hình hoặc component không còn được sử dụng nhằm hạn chế việc duy trì các kết nối không cần thiết và tránh trường hợp một màn hình nhận cùng một sự kiện nhiều lần.

Cơ chế đồng bộ này hiện phục vụ việc cập nhật dữ liệu khi ứng dụng đang hoạt động. Hệ thống hiện chưa triển khai Firebase Cloud Messaging để gửi Push Notification đến thanh thông báo Android khi ứng dụng chạy nền hoặc đã đóng.

4.3 Tích hợp và kiểm thử hệ thống

4.3.1 Tích hợp ESP32, AS608 và Firebase

Sau khi hoàn thiện phần cứng và phần mềm, ESP32, AS608 và Firebase được tích hợp thành một chuỗi xử lý thống nhất. Khi khởi động, ESP32 kiểm tra khả năng giao tiếp với cảm biến AS608 thông qua UART và thực hiện đồng bộ thời gian qua NTP khi có kết nối mạng.

Theo kết quả thử nghiệm được ghi nhận, cảm biến AS608 phản hồi thành công khi thực hiện thao tác kiểm tra kết nối và hệ thống đọc được thông tin dung lượng bộ nhớ của cảm biến, với capacity = 127 slot trong lần thử được báo cáo.

ESP32 sử dụng HTTPS REST để trao đổi dữ liệu với Firebase. Các thao tác được kiểm thử bao gồm đọc yêu cầu đăng ký, ghi bản ghi chấm công và ghi nhật ký lượt quét. Các yêu cầu được ghi nhận trả về mã phản hồi HTTP 200 OK trong các trường hợp thử nghiệm tương ứng.

Cơ chế đồng bộ thời gian NTP cũng được kiểm tra. Theo kết quả được ghi nhận trong tài liệu thử nghiệm, thời gian đồng bộ trung bình khoảng 1,2 giây kể từ khi thiết bị có kết nối Wi-Fi.

4.3.2 Tích hợp Firebase và ứng dụng Mobile

Việc tích hợp Firebase với ứng dụng di động được kiểm thử thông qua các chức năng xác thực tài khoản, phân quyền người dùng và cập nhật dữ liệu.

Với tài khoản Admin, sau khi đăng nhập hệ thống nhận diện role là admin và hiển thị giao diện quản trị với các chức năng tương ứng. Với tài khoản Employee, hệ thống nhận diện role là employee và điều hướng đến giao diện dành cho nhân viên.

Trường hợp tài khoản có trạng thái active = false cũng được kiểm thử. Khi đó, ứng dụng thực hiện đăng xuất tài khoản và hiển thị thông báo cho người dùng rằng tài

khoản đã bị vô hiệu hóa.

Cơ chế cập nhật dữ liệu chấm công cũng được kiểm tra bằng cách tạo bản ghi trên Firebase và quan sát sự thay đổi trên giao diện ứng dụng. Trong các trường hợp thử nghiệm được ghi nhận, dữ liệu chấm công được cập nhật trên ứng dụng mà không cần tải lại màn hình thủ công.

4.3.3 Kiểm thử chức năng chấm công

Các kịch bản kiểm thử được xây dựng nhằm kiểm tra các trạng thái chính của quy trình chấm công, bao gồm đúng giờ, đi muộn, Check-in, Check-out, quét trùng, vân tay chưa đăng ký và ca làm việc qua ngày.

Bảng 4.2: Kết quả kiểm thử chức năng chấm công

Mã	Tình huống và điều kiện	Kết quả ghi nhận	Đánh giá
TC-01	Vào ca lúc 07:55, ca bắt đầu lúc 08:00	Ghi giờ vào 07:55, trạng thái đúng giờ, LED xanh và Buzzer xác nhận	Đạt
TC-02	Vào ca lúc 08:20, ca bắt đầu 08:00, ân hạn 15 phút	Ghi lateMinutes = 20, trạng thái đi muộn	Đạt
TC-03	Check-in 08:00, Check-out 17:30, ca kết thúc 17:00	Ghi 9,5 giờ làm việc và 0,5 giờ tăng ca	Đạt
TC-04	Quét lại sau 60 giây, nhỏ hơn ngưỡng 300 giây	Bỏ qua lượt quét lặp, thời gian vào không bị ghi đè	Đạt
TC-05	Sử dụng vân tay chưa đăng ký	Từ chối chấm công và phát tín hiệu lỗi	Đạt
TC-06	Ca 22:00–06:00, Check-out lúc 06:15 ngày hôm sau	Ghi giờ ra vào bản ghi thuộc ngày bắt đầu ca	Đạt

Kết quả trên cho thấy các quy tắc nghiệp vụ chính của chức năng chấm công đã được kiểm tra trong các tình huống được xác định trước. Đặc biệt, hệ thống không chỉ xử lý trường hợp chấm công thông thường mà còn kiểm tra các tình huống đi muộn, quét lặp và ca làm việc qua ngày.

4.3.4 Kiểm thử chức năng ứng dụng Mobile

Đối với ứng dụng di động, kiểm thử tự động được thực hiện bằng Jest nhằm kiểm tra một số nhóm logic liên quan đến xác thực, phân quyền role, trạng thái kích hoạt tài khoản và các hàm tiện ích xử lý ngày tháng.

Kết quả được ghi nhận gồm 3 bộ kiểm thử với tổng cộng 20/20 trường hợp kiểm thử đạt, không sử dụng snapshot.

Bên cạnh kiểm thử logic, mã nguồn TypeScript được kiểm tra bằng lệnh npx tsc --noEmit và kết quả không phát sinh lỗi biên dịch TypeScript.

Quá trình đóng gói ứng dụng Android cũng được kiểm tra thông qua Gradle. Lệnh assembleDebug được thực hiện thành công với kết quả BUILD SUCCESSFUL in 31s. Thông tin APK được trích xuất bằng công cụ aapt2, trong đó tên package là com.smartattendance, dung lượng tệp khoảng 124,5 MB và tên tệp là SmartAttendance.apk.

Các kết quả trên xác nhận mã nguồn ứng dụng có thể được biên dịch và đóng gói thành APK Android. Tuy nhiên, kết quả của 20 trường hợp kiểm thử Jest chỉ phản ánh các nhóm logic đã được kiểm thử và không thể dùng để kết luận rằng toàn bộ nghiệp vụ, kết nối mạng hoặc các tình huống vận hành thực tế đều đã được kiểm tra đầy đủ.

4.4 Kết quả và đánh giá

4.4.1 Kết quả hoạt động của thiết bị chấm công

Thiết bị chấm công sử dụng ESP32 và AS608 đã thực hiện được các chức năng chính trong những lần thử nghiệm được ghi nhận. Cảm biến có khả năng nhận dạng mẫu vân tay và trả về mã mẫu để ESP32 tiếp tục xử lý nghiệp vụ chấm công.

Theo số liệu được ghi nhận trong tài liệu thực nghiệm, thời gian đọc và nhận dạng vân tay trung bình đạt khoảng 0,65 giây cho mỗi lượt quét. Tuy nhiên, do tài liệu chưa cung cấp số lượng lượt thử và điều kiện đo cụ thể, giá trị này chỉ nên được xem là

kết quả của thử nghiệm đã thực hiện, chưa đủ cơ sở để đưa ra kết luận thống kê về hiệu năng.

Các LED và Buzzer phản hồi rõ ràng đối với các trạng thái thành công và thất bại. Điều này giúp người dùng nhận biết kết quả trực tiếp tại trạm chấm công.

Ngoài ra, chức năng syncDeletedFingerprints được triển khai nhằm đối chiếu các mẫu vân tay đang lưu trên cảm biến với danh sách nhân viên còn hiệu lực trên Firebase. Các mẫu không còn liên kết với hồ sơ hợp lệ có thể được xác định và xóa khỏi bộ nhớ cảm biến.

4.4.2 Kết quả hoạt động của ứng dụng Mobile

Ứng dụng Smart Attendance được triển khai trên Android với hai nhóm người dùng là Admin và Employee trong cùng một bộ cài đặt. Các chức năng chính của hai phân hệ đã được xây dựng và kiểm thử trong phạm vi của đề tài.

Đối với Admin, ứng dụng cung cấp các chức năng quản lý nhân viên, quản lý ca làm việc, gửi yêu cầu đăng ký vân tay, xem lịch sử chấm công và thống kê báo cáo. Đối với Employee, ứng dụng cho phép theo dõi thông tin cá nhân, ca làm việc và lịch sử chấm công.

Màn hình báo cáo cung cấp các thông tin thống kê gồm tỷ lệ đúng giờ, tổng số phút đi muộn, tổng số giờ làm việc và tổng số giờ tăng ca. Các thành phần giao diện được điều chỉnh để đảm bảo tính nhất quán khi hiển thị trên Android.

Kết quả kiểm thử cũng cho thấy ứng dụng có thể được biên dịch và đóng gói thành APK với package com.smartattendance. Điều này xác nhận quá trình xây dựng ứng dụng từ mã nguồn React Native và TypeScript đến sản phẩm cài đặt trên Android đã được thực hiện.

4.4.3 Kết quả kết nối và đồng bộ dữ liệu

Luồng dữ liệu chính của hệ thống được hình thành theo trình tự từ cảm biến AS608 đến ESP32, từ ESP32 đến Firebase và từ Firebase đến ứng dụng di động.

ESP32 sử dụng HTTPS REST để gửi và cập nhật dữ liệu trên Firebase Realtime Database. Ứng dụng di động sử dụng Firebase SDK và cơ chế listener thời gian thực để nhận biết sự thay đổi của dữ liệu. Trong các tình huống thử nghiệm được ghi nhận, dữ liệu chấm công được cập nhật trên ứng dụng mà không phát hiện trường hợp trùng lặp

bản ghi trong các kịch bản đã kiểm tra.

Tuy nhiên, một số thông tin về độ trễ trong tài liệu thực nghiệm chưa có giá trị cụ thể, do đó không sử dụng các giá trị chưa được xác định để đưa ra kết luận định lượng về độ trễ của toàn hệ thống.

Cơ chế kết nối lại Wi-Fi và thử lại yêu cầu mạng giúp thiết bị có khả năng phục hồi đối với một số trường hợp gián đoạn mạng tạm thời. Tuy nhiên, hệ thống hiện chưa có cơ chế lưu tạm đầy đủ các lượt chấm công trên ESP32 để đồng bộ bù sau khi mất mạng trong thời gian dài. Vì vậy, khả năng kết nối lại hiện tại không nên được xem là cơ chế bảo đảm mất mạng mà không mất dữ liệu.

4.4.4 Đánh giá tổng thể hệ thống

Kết quả triển khai cho thấy hệ thống đã hình thành được chuỗi hoạt động chính gồm thiết bị nhận dạng vân tay, ESP32 xử lý nghiệp vụ, Firebase lưu trữ dữ liệu và ứng dụng di động hiển thị thông tin. Việc tích hợp hai vai trò Admin và Employee trong một ứng dụng giúp thống nhất quá trình triển khai và quản lý ứng dụng.

Về phần cứng, ESP32 và AS608 thực hiện được chức năng nhận dạng vân tay và phản hồi kết quả thông qua LED và Buzzer. Về phần mềm, ứng dụng React Native cung cấp các chức năng quản lý nhân viên, ca làm việc, lịch sử chấm công và báo cáo. Firebase đóng vai trò trung tâm trong việc lưu trữ và đồng bộ dữ liệu giữa thiết bị và ứng dụng.

Một điểm đáng chú ý của hệ thống là chức năng đăng ký vân tay có thể được khởi tạo từ ứng dụng quản trị. Admin tạo yêu cầu đăng ký, ESP32 nhận yêu cầu thông qua Firebase và nhân viên thực hiện thao tác trực tiếp trên cảm biến. Kết quả đăng ký sau đó được cập nhật trở lại Firebase để liên kết với hồ sơ nhân viên.

Bên cạnh những kết quả đạt được, hệ thống vẫn còn một số giới hạn. Thứ nhất, Firebase Security Rules trong phiên bản thử nghiệm mới dừng ở mức kiểm tra người dùng đã đăng nhập, do đó cần hoàn thiện phân quyền ở tầng cơ sở dữ liệu trước khi sử dụng với dữ liệu nhân sự thực tế. Thứ hai, thiết bị phụ thuộc vào kết nối Wi-Fi để gửi dữ liệu lên Firebase; cơ chế lưu tạm và đồng bộ bù khi mất mạng chưa được triển khai đầy đủ. Thứ ba, chưa có thử nghiệm tải đồng thời và thử nghiệm dài hạn đủ để đánh giá toàn diện độ ổn định khi có nhiều người sử dụng liên tục.

Đối với ứng dụng di động, hệ thống hiện hỗ trợ cập nhật kết quả chấm công khi ứng dụng đang hoạt động. Cơ chế Firebase Cloud Messaging (FCM) để gửi Push

Notification đến thanh thông báo Android khi ứng dụng chạy nền hoặc đã đóng chưa được triển khai trong phiên bản hiện tại. Đây là một hướng phát triển có thể được thực hiện trong các phiên bản tiếp theo.

Ngoài ra, kết quả thời gian quét vân tay khoảng 0,65 giây mới chỉ là kết quả thử nghiệm được ghi nhận và chưa có đủ thông tin về số lần đo, điều kiện mạng, điều kiện ngón tay và cấu hình thiết bị để thực hiện đánh giá thống kê. Tương tự, 20/20 kiểm thử Jest đạt không có nghĩa toàn bộ nghiệp vụ và các rủi ro liên quan đến mạng đã được kiểm tra.

Nhìn chung, các thử nghiệm đã xác nhận được đường đi chính của dữ liệu và các chức năng nghiệp vụ cốt lõi của hệ thống. Những giới hạn còn lại tập trung vào bảo mật dữ liệu, khả năng hoạt động khi mất mạng, thử nghiệm hiệu năng quy mô lớn và cơ chế thông báo khi ứng dụng không hoạt động. Các nội dung này được sử dụng làm cơ sở để đề xuất hướng phát triển trong chương tiếp theo.

CHƯƠNG 5: KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN

5.1 Kết luận

Đề tài "Xây dựng ứng dụng di động chấm công bằng vân tay kết hợp ESP32 và Firebase" đã được triển khai và hoàn thành theo đúng các mục tiêu nghiên cứu và yêu cầu kỹ thuật đặt ra trong môn học Phát triển ứng dụng di động. Hệ thống đã xây dựng thành công một mô hình quản lý chấm công khép kín, phân tầng rõ ràng từ phần cứng ngoại vi đến hệ thống lưu trữ đám mây và ứng dụng di động phía người dùng.

Những kết quả chính mà đề tài đã đạt được bao gồm:

5.1.1 Về phần cứng và hệ thống nhúng

Xây dựng thành công trạm chấm công sử dụng vi điều khiển ESP32 NodeMCU LuaNode32 kết hợp cảm biến vân tay quang học AS608. Hệ thống đảm bảo khả năng thu nhận sinh trắc học, xử lý nhận dạng qua giao tiếp UART, đồng thời cung cấp hệ thống chỉ báo trực quan tại chỗ thông qua LED xanh, LED đỏ và còi Buzzer Active.

5.1.2 Về cơ sở dữ liệu và đồng bộ

Tận dụng thành công các dịch vụ của Firebase (Firebase Authentication, Realtime Database, Storage) để xây dựng hệ thống lưu trữ tập trung. Cơ chế đồng bộ dữ liệu theo thời gian thực (Realtime Listener) giúp ứng dụng di động tự động cập nhật trạng thái chấm công khi có sự kiện mới phát sinh mà không yêu cầu người dùng tải lại màn hình thủ công.

5.1.3 Về ứng dụng di động

Phát triển thành công ứng dụng Smart Attendance bằng React Native và TypeScript trên nền tảng Android. Ứng dụng tích hợp hai nhóm phân quyền người dùng (Admin và Employee) trong cùng một bộ cài đặt, đáp ứng đầy đủ các nghiệp vụ trọng tâm như xác thực tài khoản, quản lý hồ sơ nhân viên, phân công ca làm việc, đăng ký vân tay từ xa, tra cứu lịch sử chấm công và trích xuất báo cáo thống kê.

5.1.4 Về kiểm thử và độ tin cậy

Hệ thống đã được kiểm thử qua nhiều kịch bản nghiệp vụ thực tế (chấm công đúng giờ, đi muộn, xử lý ca làm việc qua ngày, cơ chế chống quét lặp) cũng như kiểm

thử logic mã nguồn bằng Jest và đóng gói thành công tệp cài đặt APK (com.smartattendance).

Thông qua quá trình thực hiện, đề tài đã tạo cơ hội để vận dụng linh hoạt các kiến thức về lập trình nhúng, thiết kế cơ sở dữ liệu đám mây và kỹ thuật phát triển ứng dụng di động đa nền tảng vào việc giải quyết một bài toán quản lý thực tế có tính ứng dụng cao.

5.2 Các hạn chế còn tồn tại

Mặc dù hệ thống đã cơ bản đáp ứng các yêu cầu chức năng, phiên bản triển khai hiện tại vẫn còn một số điểm hạn chế nhất định do giới hạn về thời gian và phạm vi nghiên cứu:

5.2.1 Về bảo mật và phân quyền dữ liệu

Quy tắc bảo mật dữ liệu trên Firebase Realtime Database (Firebase Security Rules) trong giai đoạn thử nghiệm mới chỉ dừng lại ở mức xác thực người dùng đăng nhập (isAuthenticated). Hệ thống chưa thiết lập chi tiết các quy tắc kiểm soát quyền đọc/ghi theo từng vai trò (Admin/Employee) tại tầng cơ sở dữ liệu.

5.2.2 Về tính ổn định khi mất kết nối mạng

Trạm phần cứng ESP32 phụ thuộc hoàn toàn vào kết nối Wi-Fi để truyền dữ liệu lên Firebase theo thời gian thực. Hệ thống hiện chưa triển khai cơ chế lưu trữ tạm thời (buffer/offline storage) các bản ghi chấm công trên bộ nhớ trong của ESP32 để thực hiện đồng bộ bù (sync buffer) khi mạng Internet bị gián đoạn trong thời gian dài.

5.2.3 Về cơ chế thông báo trên ứng dụng di động

Ứng dụng hiện tại chỉ cập nhật giao diện khi ứng dụng đang mở và hoạt động nhờ cơ chế Realtime Listener. Hệ thống chưa tích hợp dịch vụ Firebase Cloud Messaging (FCM) để gửi thông báo đẩy (Push Notification) trực tiếp đến thanh thông báo của thiết bị di động khi ứng dụng ở trạng thái chạy nền hoặc đã đóng.

5.2.4 Về quy mô kiểm thử

Các kết luận về hiệu năng (như thời gian phản hồi của cảm biến hay độ ổn định mạng) mới chỉ dựa trên các lượt thử nghiệm độc lập ở quy mô nhỏ, chưa thực hiện kiểm

thử tải đồng thời (load testing) với số lượng lớn người dùng truy cập hoặc quét vân tay liên tục trong cùng một thời điểm.

5.3 Hướng phát triển trong tương lai

Dựa trên những kết quả đã đạt được và các hạn chế còn tồn tại, hệ thống có thể được mở rộng và hoàn thiện hơn trong các phiên bản tiếp theo thông qua các hướng phát triển cụ thể sau:

5.3.1 Hoàn thiện hệ thống bảo mật

Xây dựng và áp dụng bộ quy tắc Firebase Security Rules chặt chẽ nhằm phân quyền chi tiết quyền đọc và ghi dữ liệu cho từng node cụ thể, đảm bảo tính bảo mật tuyệt đối cho thông tin nhân sự và dữ liệu chấm công.

5.3.2 Nâng cấp khả năng hoạt động ngoại tuyến (Offline Mode)

Bổ sung thuật toán lưu trữ tạm thời các lượt chấm công vào bộ nhớ Flash hoặc EEPROM của ESP32 khi mất kết nối mạng, đồng thời thiết lập cơ chế tự động đồng bộ bù dữ liệu lên Firebase ngay khi thiết bị kết nối lại mạng Internet.

5.3.3 Tích hợp thông báo đẩy (Push Notification)

Triển khai Firebase Cloud Messaging (FCM) để gửi thông báo tức thời đến điện thoại của nhân viên ngay khi hệ thống ghi nhận thành công lượt Check-in hoặc Check-out, kể cả khi ứng dụng đang chạy nền hoặc đã bị tắt.

5.3.4 Mở rộng nền tảng và kiểm thử hiệu năng

Tiến hành kiểm thử ứng dụng trên nền tảng iOS, thực hiện kiểm thử tải (Load Testing) toàn diện đối với cụm cơ sở dữ liệu Firebase và trạm phần cứng nhằm đánh giá độ bền vững của hệ thống trước khi đưa vào ứng dụng vận hành thực tế tại các doanh nghiệp.